Preprint, not peer reviewed

ThoxWork: One Workplace, One Connected Team. A Policy-Bounded DigitalHuman Work Platform for Enterprise Use

ThoxWork is the third THOX platform: the work-organization layer in which a company assigns real work to named DigitalHuman colleagues (ThoxEmployees), reaches them through Rolodex, bounds their authority and resources by MeshGroup, and draws on approved capacity through MeshStack. This comprehensive edition defines the model formally, audits the code built toward it, evaluates the model's invariants in a deterministic virtual lab, measures the rules against the product's own code, and replays the full evaluation against the live implementation.

THOX.ai Research · Authors: Craig Ross, Chief Executive Officer (ORCID 0009-0005-9872-3673), and Tommy Xaypanya, CTO and Co-Founder (corresponding author; ORCID 0009-0000-4425-9565), THOX.ai LLC

ABSTRACT

Enterprise adoption of AI agents is outrunning enterprise governance of them. Vendor surveys report organizations experimenting with agents at scale while only a minority operate policies to secure them, workplace benchmarks show the best agents completing a minority of consequential tasks unaided, and orchestration research shows that at team scale the deciding factors are not model capability but discovery scope, handoff fidelity, and the structure of human review. THOX's answer is a work platform, not another agent framework. In the ThoxWork model, work lives in shared workspaces; DigitalHumans are named, role-holding colleagues who can be assigned tasks and held to account for them; Rolodex is the team interface through which a person assigns a task, checks in, and reviews results; a MeshGroup binds a named team to the exact set of resources it is approved to use; and MeshStack connects those approved resources. This edition reports evidence in three labelled tiers. **Simulated:** the model's five invariants (unapproved resources never run; oversized work is denied rather than spilled; reassignment preserves a complete handoff chain; no task reaches Done without eligible human review; queues complete predictably as teams and groups scale) are evaluated in a deterministic, seeded virtual lab, where every pre-registered bar is met. **Measured, code and lab:** an evidence lab re-implements the product's routing planner and the reference admission kit and agrees with both on every seeded case (12,000 of 12,000 previews; 20,000 of 20,000 kit decisions); a divergence study re-run under the paper's own admission semantics shows pooled admission admitting 3,882 of 20,000 probed requests that no single node could serve, and quantifies how a looser port of the same rule over-admits; and a re-implemented detection matrix shows what a keyed, externally anchored audit chain detects that hash links alone cannot. **Measured, live implementation:** the same evaluation generators that produced the simulated results were replayed against thoxwork-core, the live Rust implementation of the model, and every bar was met again on the running system, including the exact 120 admitted / 121 UNPLACEABLE capacity boundary and a three-judge agreement check over 102 workspaces. The edition also reconciles the record it found: three admission definitions and five assignment state machines are shown side by side, the one material disagreement is dispositioned, and the platform's upstream lineage (OpenWork, OpenCode, Hermes Agent) is stated with its license posture. ThoxWork as a purchasable enterprise platform remains in development; THOX's own product copy says so, and this paper claims nothing further.

SIMULATED · PROTOCOL ORACLE, SECTION 7.1 TO 7.5 (E1 TO E5)

0 / 12,000
Unapproved-resource attempts leaked across direct, cross-group and escalation attacks (E1)
SIMULATED

1.000
Handoff-chain completeness over 5,000 reassigned tasks and 15,000 chain entries; 0 of 4,000 negative probes leaked (E2)
SIMULATED

100%
Of 4,000 oversized tasks denied UNPLACEABLE; 0 silently placed, 0 spilled to another group (E3)
SIMULATED

0 / 6,000
Review-gate bypasses succeeded across six attack classes; 4,000 of 4,000 legitimate tasks completed through review (E4)
SIMULATED

MEASURED · LIVE IMPLEMENTATION REPLAY, SECTION 7.11 (E11)

0 / 12,000
Unapproved-resource attempts leaked against the live thoxwork-core API, same generators, same seeds (E11)
MEASURED · LIVE

120 / 121
Admission boundary on the live system: demand 120 RU admitted, 121 RU denied UNPLACEABLE, in every replicate (E11)
MEASURED · LIVE

0 / 6,000
Live review-gate bypasses; the gate-off control arm completed 4,000 of 4,000 without review, so the check discriminates (E11)
MEASURED · LIVE

102 / 102
Workspaces on which three independent judges (verifier API, verifier CLI, harness checker) agree; fleet sweep: 121 workspaces, 0 non-clean (E11)
MEASURED · LIVE

MEASURED · CODE AND LAB, SECTIONS 7.6 TO 7.10 (E6 TO E10)

12,000 / 12,000
Seeded routing previews on which the lab planner and the product planner agree on every compared field (E6)
MEASURED · CODE

20,000 / 20,000
Seeded decisions on which the lab module and the reference admission kit agree, including every tie (E7)
MEASURED · CODE

3,882
Of 20,000 probed requests admitted by pooled admission (Algorithm 1) that no single node could serve; the strict single-node rule refuses all of them (E8)
MEASURED · LAB

500 / 500
Tail truncations of the audit chain detected with an external anchor; 0 of 500 detected without one, keyed or not (E9)
MEASURED · LAB

Status at writing, 2026-11-04. This is edition v1.1.0, the comprehensive public edition under the THOX Research Paper Process v1.0; it supersedes v1.0.1 (DOI 10.5281/zenodo.23963936) and carries its simulated results forward unchanged. Three evidence tiers are used and labelled everywhere: the deterministic protocol oracle (`exp/results/summary.json`, SHA-256 `f7c72f64b9b8c…`, re-verified byte-identical for this edition); the code and lab measurements of Sections 7.6 to 7.10, produced from the delivered artifacts of the comprehensive companion review and, for E8 and E9, re-run under this paper's semantics and independently re-verified; and the live replay of Section 7.11 against thoxwork-core (acceptance run tag `f182809`). Nothing in this paper is measured on a customer deployment, and no customer, deployment count, or commercial term is claimed anywhere in it.

CONTENTS

1. Introduction: the third platform
2. Background, related work and positioning
3. System and threat model
4. The ThoxWork model, reconciled
5. What is built: an evidence-bounded account
6. Method: three instruments
7. Evaluation
8. Governance mapping
9. Reading the results
10. Product implications
11. Security considerations
12. Limitations

13. Decisions and roadmap
14. Provenance, consent and document control
15. Appendix A. Reproducibility
16. Appendix B. Notation
17. Appendix C. Errata register
18. Appendix D. Disclosure boundary
19. References

1 Introduction: the third platform

THOX builds local-first AI in layers. Its devices and operating system put capable models on hardware the customer owns [77]. Its MeshStack fabric lets approved devices pool their capacity for each other, under grant-gated, lease-fenced rules that have been published and measured elsewhere [2]. Neither layer, by itself, answers the question a company actually asks on a Monday morning: who is doing this work, what are they allowed to touch, who approved the consequential step, and how do we know it is done?

ThoxWork is the layer that answers that question. It is THOX's third platform: the work-organization and governance layer for companies that employ DigitalHumans alongside people. Its positioning, fixed in THOX product records in October 2026 and reaffirmed in November 2026 planning, is one sentence long: One workplace. One connected team. ThoxWork organizes the work. Rolodex connects you to your DigitalHuman team. MeshGroups define team access. MeshStack connects approved resources.

Workplace benchmarks explain why the answer cannot be left to the agents. The best agent in TheAgentCompany completes 30 percent of tasks in a simulated software company on its own [3]; on OSWorld the best model reaches 12.24 percent against 72.36 percent for people [4]; tau-bench finds a strong model succeeding on under half of its tasks, with pass^8 below 25 percent in retail [5]. Multi-agent systems add their own failure modes: MAST annotated more than 1,600 traces across seven frameworks and placed inter-agent misalignment and task verification among its three failure categories [6]. A platform for agent work therefore has to ship review, scoping and records as mechanisms, not as advice.

The vocabulary matters, because each term names a distinct responsibility that other approaches merge or omit:

- **DigitalHumans** are named, role-holding synthetic colleagues. A DigitalHuman has an identity, a role, a team, and a bounded remit. Under THOX's DigitalHuman Colleague and Disclosure Standard, they are treated and credited as colleagues, not tools [75].
- **ThoxEmployees** is the workforce offering built on DigitalHumans: digital employees an organization staffs into roles, as distinct from a chatbot a person prompts.
- **Rolodex** is the team interface: the place a person finds their DigitalHuman team, assigns a task, checks in on it, and reviews the result.
- **A MeshGroup** is the unit of bounded authority: a named set of team members plus the exact set of resources that team is approved to use.
- **MeshStack** is the resource fabric a MeshGroup draws on: approved models, files, projects and results, connected under the published MeshStack rules [2].

- **ThoxWork** is the platform that holds these together: shared workspaces, assignments, policy, and the evidence trail of who did what, reviewed by whom.

The first edition of this paper stated the model as five testable invariants and evaluated them in a deterministic virtual lab, deliberately leaving the code out of scope [1]. A buyer, an auditor or a peer reviewer asks the next question: does the built code implement the stated rules, and where the records disagree, which record wins? This comprehensive edition answers that question, and then goes one step further: it replays the lab's own evaluation against the live implementation of the model. Its contributions are:

- **The model, reconciled (Sections 3 and 4).** One vocabulary across five assignment state machines, three MeshGroup admission definitions shown side by side rather than merged, a fail-closed action gateway stated as an algorithm, and an approval rule stated as an equation. The one material disagreement the reconciliation surfaced, between the reference admission kit and the pooled admission rule of the first edition, is dispositioned in Section 4.4 on implementation evidence.
- **An evidence-bounded account of what is built (Section 5).** Ten THOX code bases at stated commits, 3,747 tests rerun with 3 failures, the platform's upstream lineage stated with its license posture, and status labels that separate shipped, preview, designed and proposed components.
- **Three instruments, three evidence tiers (Section 6).** A deterministic protocol oracle (SIMULATED), a browser evidence lab checked for agreement against the product's own code (MEASURED, code and lab), and the live thoxwork-core implementation replayed with the oracle's own generators (MEASURED, live).
- **Measurements (Section 7).** Planner agreement, kit agreement, an admission-divergence study re-run under the paper's own semantics with the WAIT path exercised, an audit-chain detection matrix re-implemented from the model's evidence equation, decision latency, and the live replay as the centre of gravity.
- **A governance mapping and an errata register (Section 8, Appendix C).** The mechanisms mapped to NIST, ISO and EU requirements as alignment, not conformance; and every correction this edition makes to the first edition's text, with the source each correction was checked against.

2 Background, related work and positioning

Every property in the ThoxWork model has close prior work. The contribution, if there is one, is where the properties meet, so this section names the nearest precedent for each and states the difference. None of the cited work evaluates ThoxWork; each item is used as a design warrant, and the distinction is maintained throughout.

2.1 Enterprise context

Adoption is ahead of governance. McKinsey's State of AI 2025 (N = 1,993, 105 countries) reports 62 percent of organizations at least experimenting with agents, 23 percent scaling an agentic system in at least one function, and no more than 10 percent scaling agents in any given function [7]. A 2025 survey of 353 IT professionals found 82 percent of organizations already using AI agents and 44 percent with

policies in place to secure them [8]. Microsoft's 2026 Work Trend Index (N = 20,000) reports organizational factors carrying roughly twice the permutation importance of individual factors for self-reported impact, an associational result rather than a causal one [9]. MIT CISR describes digital colleagues as agents managed like employees and reports that 75 percent of 132 surveyed organizations predicted a 25 percent average rise in revenue per employee over three years [10]. Standards bodies are moving: NIST launched an AI Agent Standards Initiative [11] and an NCCoE concept paper on agent identity and authorization [12], and the Cloud Security Alliance proposed an Agentic Trust Framework with maturity levels from Intern to Principal [13, 14]. The Model Context Protocol and the Agent2Agent protocol, announced by Anthropic and Google [15, 16], are now stewarded under the Linux Foundation [17, 18]; they standardize agent-to-tool and agent-to-agent traffic, not the workspace where people and agents share accountable work.

2.2 Multi-agent systems and agent workplaces

Role-based multi-agent frameworks established that assigning LLM agents distinct roles and passing structured work between them improves results: CAMEL's role-playing agents [19], MetaGPT's standard operating procedures for an assembly line of roles [20], ChatDev's chat chain across a software company [21], AutoGen's conversable agents [22] and AgentVerse's dynamic expert groups [23]. Generative Agents showed persistent identities with memory and reflection [24]. Surveys organize the field [25, 26, 27, 28], and Magentic-One coordinates specialist agents under an orchestrator [29]. These systems run handoffs inside one autonomous session; none records each handoff as an approvable, verifiable artifact, and none scopes resources by team. In a controlled simulation of event-driven orchestration across registries of under 10, 20 to 80 and 200 agents, both tested systems scored lower at enterprise scale; the authors suggest a larger and more confusable discovery space may contribute, and a minimal task manager cut high-priority queue latency by 14 to 75 percent [30]. DynaFlow learns free-form orchestration programs that make the hand-off between stages explicit [31], reaching its gains by training only the orchestrator to emit executable handoff programs; cross-stage information flow is the deciding factor, which is the design warrant for ThoxWork's written, hashed handoff chain (Definition 3). Benchmarks of agents at work include TheAgentCompany [3], WorkArena [32], CRMArena [33], OSWorld [4] and tau-bench [5]. CorpGen simulates whole corporations staffed by autonomous digital employees and is the closest use of that term in the literature, as a benchmark rather than a governed product [34].

2.3 People in the loop

Magentic-UI's action guards are the closest published analogue to an approval gate, built for one user and one task [35]. Feng, McDonald and Zhang define five levels of agent autonomy by the user's role, one of which is approver [36]; Cocoa lets plan steps be assigned to the person or the agent [37]. Collaborative Gym reports that collaborative agents beat autonomous ones with real users at 86, 74 and 66 percent win rates across three tasks [38]. Field evidence points the same way: in a randomized experiment that assigned 2,234 participants to human-human and human-AI teams producing 11,024 advertisements, human-AI teams produced 50 percent more ads per worker, and their collaboration was more task-oriented and showed more delegation [39]; a second preregistered experiment with

1,258 participants found that pairing agent and human personalities changed output quality, which bears on how named personas are presented [40]. The Collaborative Human-Agent Protocol (CHAP) makes the human override a structured event with a diff, a rationale and a content hash, turns a shift handoff into a portable envelope, and makes an approval a signed decision that can be replayed [41]. It is the closest specification to the ThoxWork team layer; its abstract does not describe admission of work against approved resources, which is where MeshGroups differ.

2.4 Governance, regulation and security

Standards converge on a named, competent person who can stop or reverse an AI system. NIST AI 100-1 asks organizations to define roles for human-AI configurations and oversight (GOVERN 3.2) and to document oversight processes (MAP 3.5) [42]; the generative AI profile adds governance, content provenance, pre-deployment testing and incident disclosure as considerations [43]. ISO/IEC 42001 is a management-system standard for organizations that provide or use AI [44]. The EU AI Act requires record-keeping (Article 12), effective human oversight including the ability to override, reverse or stop (Article 14), and deployer log retention of at least six months (Article 26(6)) for high-risk systems [45]; the Digital Omnibus on AI moved the high-risk application date for Chapter III Sections 1 to 3 to 2 December 2027 [46]. A general workspace is not high-risk by default, but Digital Employees that allocate tasks to people or evaluate their performance would bring a deployment into Annex III scope.

OWASP names excessive agency (LLM06) and prompt injection (LLM01) among the top risks for LLM applications and prescribes least privilege, human approval for high-impact actions and complete mediation outside the model [47]; its agentic guidance and 2026 top ten add tool misuse, identity and privilege abuse, and human-agent trust exploitation [48, 49]. Indirect prompt injection turns retrieved content into instructions [50]; AgentDojo and InjecAgent measure how often agents follow them [51, 52]. In a multi-agent workspace a handoff moves content from one agent's context into another's, so the access check has to happen again at each handoff and at each group boundary. NIST zero trust guidance grants access per session with least privilege and states that authorization to one resource does not grant another [53]; attribute-based access control supplies the policy model [54]. South et al. extend OAuth and OpenID Connect to delegated agent credentials [55], and Chan et al. argue for agent identifiers, activity logs and infrastructure for accountability [56, 57].

2.5 Provenance and tamper-evident records

Secure audit logs that an attacker can neither read nor undetectably modify date to Schneier and Kelsey [58]. Crosby and Wallach showed that tree-structured logs give compact membership and consistency proofs where a plain hash chain needs a linear scan [59]; Certificate Transparency put those proofs into production for public logs [60, 61]. W3C PROV defines entities, activities, agents and delegation [62, 63]; in-toto verifies signed per-step attestations against a declared layout [64]; SLSA defines build provenance [65]. For agents, PROV-AGENT extends PROV to agent interactions [66], Auditable Agents defines five auditability dimensions and reports 8.3 ms median overhead for mediated, tamper-evident execution [67], and a September 2026 study shows that agents can delete their own traces, so logging must sit outside the agent's control [68]. The primitives are SHA-256 [69]

and HMAC [70]. Local-first software keeps the primary copy of data with its owner [71], which is where THOX places workspace records.

2.6 Positioning

System	Persistent role identity	Human approval gate	Recorded ordered handoffs	Team-scoped resources	Tamper-evident record
Generative Agents [24]	yes	not stated	not stated	not stated	not stated
MetaGPT [20]	partial	not stated	partial	not stated	not stated
ChatDev [21]	partial	not stated	partial	not stated	not stated
AutoGen [22]	partial	partial	not stated	not stated	not stated
Magentic-UI [35]	not stated	yes	not stated	not stated	not stated
CorpGen [34]	yes	not stated	not stated	partial	not stated
Authenticated Delegation and Authorized AI Agents [55]	not stated	partial	not stated	partial	partial
PROV-AGENT [66]	not stated	not stated	partial	not stated	partial
Auditable Agents [67]	not stated	not stated	not stated	not stated	yes
CHAP [41]	not stated	yes (signed decisions)	yes (handoff envelopes)	not stated	partial (content hashes)
ThoxWork, built today	yes (named employees with sponsors)	yes (person-only, role-named)	partial (per-handoff body hash)	yes, preview-only (MeshGroups)	partial (hashes detect accidental change)
ThoxWork with the model's evidence design	yes	yes	yes (chained, keyed)	yes	yes (keyed chain plus external anchor)

Table 1. Positioning against the closest prior work. Entries for other systems are judged from their abstracts only ("not stated" means the abstract does not say, not that the system lacks it). ThoxWork rows are judged from the code and the live implementation reviewed for this paper (Sections 5 and 7.11).

No single prior system combines the five properties, and each property alone has precedent. The defensible claims are therefore about the seams: an approval stored in the same tamper-evident record as the action it authorizes, an access check repeated at every handoff and every group boundary, and evidence written by the workspace rather than by the agent [68]. Against central cloud platforms and open multi-agent frameworks, the ThoxWork model additionally takes three stated positions: its unit of authorization is the MeshGroup's approved resource set, evaluated by an admission rule with a placeability condition (CHAP does not specify resource admission); its default data posture is local-first rather than server-hosted [71]; and its team members include DigitalHumans held out as named colleagues under an explicit colleague standard [75], which makes the Rolodex directory a staffing surface (who holds which role, at what tier) and not only a discovery surface. No ratified agent-governance standard exists at writing, and this paper makes no conformance claim.

3 System and threat model

Actors. People (members, sponsors, reviewers and approvers), Digital Employees (synthetic colleagues with a role, an autonomy level and a human sponsor), the ThoxWork workspace and its server, the ThoxEmployee runtime, and the nodes and sites that serve models. A MeshGroup binds actors to workspaces, model pools and nodes.

Assets. Workspace data at a stated data class; the authority to cause an external effect (send, write, execute, call a tool); the record of who did what and who approved it; and node capacity owned by the organization or by individual device owners.

Adversaries and failure sources. (A1) a Digital Employee following injected instructions from retrieved content [50]; (A2) a member or employee reaching resources bound to another group; (A3) an insider or compromised process with write access to stored records, including one that recomputes unkeyed hashes; (A4) a stale or forged node observation that makes an unavailable or unverified node look eligible; (A5) a careless or rubber-stamp approval (automation bias, Article 14(4)(b) [45]); (A6) an agent that deletes or rewrites its own trace [68]. Out of scope: compromise of the host operating system, of the person's own credentials, and of the model weights.

Requirements. The paper evaluates the design against eight requirements.

Id	Requirement	Statement
R1	Fail closed	An action not explicitly known and permitted is denied; a tenant cannot widen a deployment ceiling.
R2	People decide consequential steps	An external effect waits for a person when any approval reason holds; an agent cannot approve; each request names and records the role required to decide it; a decision is final.
R3	Legal lifecycle	An assignment moves only along its contract's transitions; every refused move is recorded.
R4	Team-scoped access	A request is planned only within its MeshGroup's bindings and the member's explicit actions; cross-site and remote use need every policy level to allow it.
R5	Admission honesty	Work that cannot be served is refused with a reason, never silently routed elsewhere.
R6	Recorded handoffs	Each change of hands leaves a written handoff that a reviewer can verify.
R7	Tamper-evident record	Editing, reordering, deleting, truncating or rebuilding the record is detectable by a party that did not write it.
R8	Contradictions surface	Conflicting facts are kept and cross-tagged, never merged or dropped.

Table 2. Requirements used in Sections 4 to 7.

4 The ThoxWork model, reconciled

The model is stated here at the level of rules, not implementation. Implementation mechanisms that are not already published are THOX trade secrets and are out of scope for this paper; the rules below are the public contract the platform is evaluated against in Sections 6 and 7. This section reconciles the first edition's model [1] with the code reviewed for this edition, and keeps every disagreement visible rather than merging it away.

FIGURE 1 · THE THOXWORK STACK. EACH LAYER HAS ONE RESPONSIBILITY; AUTHORITY NARROWS DOWNWARD, EVIDENCE FLOWS UPWARD.

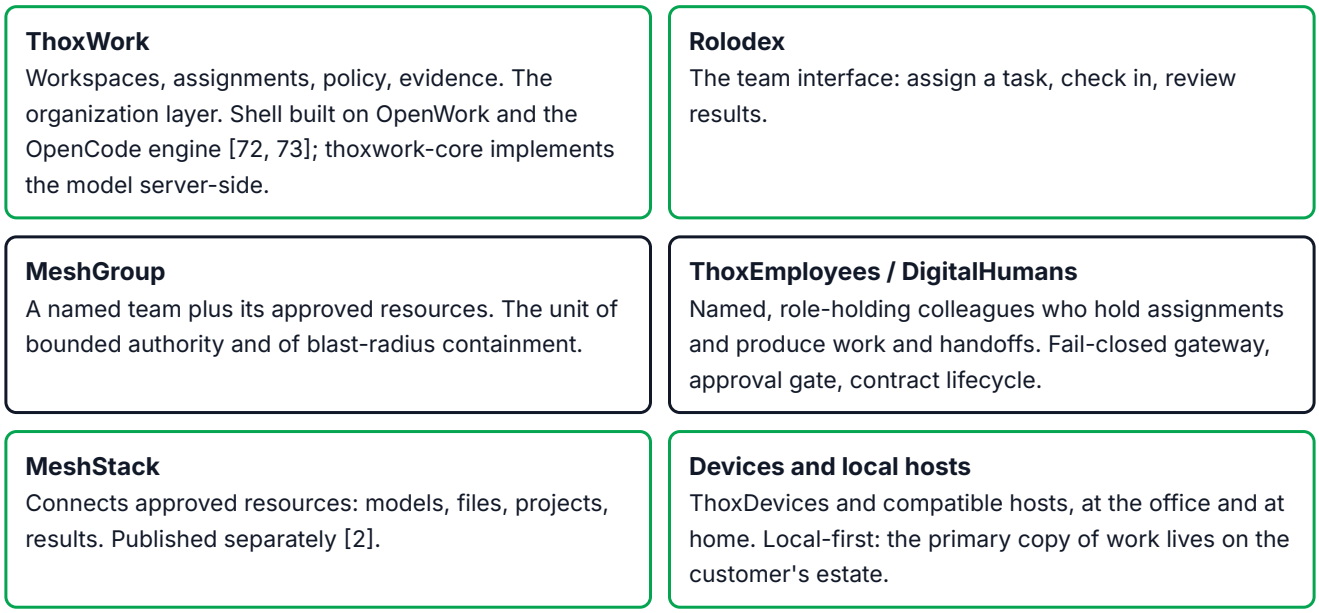


Figure 1. Layer responsibilities in the ThoxWork model. The diagram states design positions, not deployment measurements.

4.1 Definitions and invariants

Definition 1 (MeshGroup). A MeshGroup G is a pair (M, R) of a member set M (people and DigitalHumans) and an approved resource set R. The pooled approved capacity of G is the sum of the capacities of the resources in R. There is no global pool: capacity, membership and approval are properties of a group, never of the platform as a whole.

Definition 2 (Task). A task belongs to exactly one MeshGroup, declares the resources it requires and an abstract size (its demand), and carries exactly one assignee at a time. A task's required resources and demand are part of its record and are visible to its group.

Definition 3 (Handoff chain). Every reassignment of a task appends exactly one entry to its handoff chain: sequence number, previous holder, new holder, time, a non-empty written handoff note, and a hash of that note. Entries are append-only. A chain is complete when sequence numbers are contiguous, each entry's new holder is the next entry's previous holder, the final holder is the current assignee, and every note is present and hashes correctly.

Definition 4 (Legitimate completion). A task is legitimately Done when its record contains a review event in which an eligible reviewer, who is a member of the task's MeshGroup, is not the assignee, and holds a reviewer or manager role, approved the task while it was In Review, and the task entered Done in that same transition.

Definition 5 (Admission). A task is admitted to run only under the admission rule of Algorithm 1, evaluated when the task is created and again when it is assigned, so that approval cannot be checked

once and exploited later. The pool of a group G for a resource r is the sum of free resource units over the devices in G that run r; admission is a property of the pool, never of a single device.

The model stands or falls on five claims, each stated so it can be falsified:

1. **I1, policy enforcement.** A task that requires a resource its MeshGroup has not approved can never run with that resource attached, directly, through another group's approval, or by adding the resource after admission.
2. **I2, handoff integrity.** Every reassignment preserves a complete handoff chain (Definition 3); silent reassignment is impossible, including reassignment with an empty note, to a non-member, or out of In Review.
3. **I3, admission honesty.** Work larger than its group's pooled approved capacity is denied UNPLACEABLE in the tick it is evaluated. It is never placed partially, split, run on another group's resources, or left queued forever to disguise the denial.
4. **I4, the review gate.** No task reaches Done except by legitimate completion (Definition 4). Convenience paths, self-review, wrong-role review, non-member review and stale approvals all fail.
5. **I5, predictable scale.** Under a stated simulated-latency model, queues complete as team size and MeshGroup count grow; throughput does not collapse. I5 is a claim about the model's behavior under its stated assumptions, and Section 9 is explicit about how much weight it can bear.

4.2 The action gateway

A Digital Employee has an identity, a role, a sponsor and an autonomy level. In ThoxEmployee every gated action passes a policy gate before it can take effect, and the gate fails closed (R1). Algorithm 2 gives the order of evaluation as built; the first rule that decides is recorded with the decision, and an allowed effect is wrapped in receipts: a running entry before it and its outcome after. A write or execute action can be previewed first; the preview yields a digest of the request and the policy in force, and an execution that presents a digest for different content is refused.

Algorithm 2 · Action gateway (fail closed, as built)

```
gate(actor, action, payload, channel):
 1. on the deployment deny list     → DENY deployment_deny
    (tenant policy cannot widen it)
 2. not a known action              → DENY unknown_action
 3. on the tenant deny list         → DENY deny_actions
 4. actor not on channel roster     → DENY channel_membership
 5. payload has a secret marker     → DENY secret_leak
 6. payload over the size cap       → DENY size_limit
 7. command execution disabled,     → DENY exec_disabled,
    empty or not allowlisted           exec_empty, exec_allowlist
 8. otherwise                       → ALLOW default_allow
    (running receipt before the effect, outcome after)
```

4.3 The approval gate and the assignment lifecycle

Work assigned to a Digital Employee is an assignment under the Digital Employee contract. Before any external effect, the assignment requires approval when any of four reasons holds (Equation 1): the

caller kept the external-effect gate on, which is the default; a responsibility of the role requires a person; the data sensitivity is confidential or restricted; or the effective autonomy, the lower of employee and role autonomy, is zero.

$$\text{needs_approval}(a) = g_a \vee \exists r \in \text{resp}(a): h_r \vee \sigma_a \in \{\text{confidential}, \text{restricted}\} \vee \min(\alpha_e(a), \alpha_r(a)) = 0 \quad (1)$$

Only a person can decide. Subjects that identify an employee, agent, assignment or service are refused; each request names and records the role required to decide it, which rises with sensitivity, and the Digital Humans gateway checks that role before it accepts a decision; a second decision on the same request is refused. Approval returns the assignment to running; rejection blocks it. The contract defines seven states and eighteen legal transitions (Table 3); every other move is refused before any event is emitted, and every change increments a version used for optimistic concurrency.

From	Allowed to
queued	running, blocked, failed, cancelled
running	awaitingApproval, blocked, completed, failed, cancelled
awaitingApproval	running (approve), blocked (reject), failed, cancelled
blocked	queued, running, failed, cancelled
completed	terminal
failed	queued (retry)
cancelled	terminal

Table 3. Assignment lifecycle of the Digital Employee contract: 7 states, 18 legal transitions.

Five state machines describe work across the THOX repositories: the contract above, a gateway review inbox, the ThoxEmployee review workspace, the ThoxEmployee kernel's task states and Migration Studio's plan-first states. Rolodex presents four user states and one exception state (Figure 2). Table 4 maps them onto the contract. One disagreement matters: the contract maps rejection to blocked, while the gateway's mapping sends it to cancelled. The contract is the authority; Section 13 records the alignment decision.

Rolodex (this paper)	Contract v2	Review workspace	Kernel task
Queued	queued	queued	queued
In Progress	running	running	busy
In Review	awaitingApproval	awaiting_approval	(no human queue; operator-replayed preview digest)
Done	completed	completed	done
Changes requested (back to In Progress)	blocked, then running	rejected, then queued	blocked
Denied (terminal, at admission)	refused at admission; no assignment created	not modelled	blocked (policy refusal)

Table 4. One vocabulary across the assignment state machines found in the code. The gateway inbox and Migration Studio follow the same pattern with their own names.



Figure 2. The Rolodex state machine. The user's loop is three verbs: assign a task, check in, review results. Check-in is a progress act inside In Progress; it never advances a task to Done and never substitutes for review. Request-changes returns a task from In Review to In Progress. Reassignment is permitted from Queued and In Progress, never from In Review, Done or Denied, and always requires the written handoff note of Definition 3. Denied is reachable only from admission (Algorithm 1) and is terminal.

4.4 Rolodex, ordered handoffs and validated dispatch

In this paper Rolodex is the loop through which a person assigns work, checks in and reviews it. In the code, the name covers three different things, and this paper keeps them apart. The **Rolodex directory** is a web app over the nine customer-facing Digital Employees: an ID card for each, a recorded introduction, chat with one employee at a time after an explicit consent to hosted processing, and a team builder whose output is a selection marked for review that grants no tools and no permissions. It has no ordered task and no handoff. **RolodexGo** is a native client for iPhone, iPad and Mac that pairs with a private server and carries the same roster; Section 5 states its landing status precisely. The flow the public site describes as building a team, running an ordered task and reviewing written handoffs is the **ThoxEmployee guest demo**, which reuses the ThoxEmployee workspace engine.

The guest demo's engine is the clearest built example of the assign, check-in and review loop. A person composes a task for a small team (the interface allows up to three employees), always with approval required. The engine runs one step per employee in order, and each step receives every earlier handoff as context. A step can only claim the next pending position, every write is fenced by a revision check, and each run and step carries an idempotency key, so a retried or duplicated step cannot double-run or skip ahead. Each step leaves a written handoff in the audit trail, the task then waits for a person, and approve or reject is a recorded decision that performs no external action. The hosted demo adds daily call limits per session, per network address and in total, and expires a guest session after 24 hours. Table 4's first column is this loop's vocabulary.

Validated dispatch is the pattern THOX uses where one answer is assembled from several workers. A request passes input guardrails, is planned, executed by workers in parallel, cross-checked and synthesized, then judged by a validator in two layers: fixed rules first, which send obviously empty or refused output back for revision without a model call, and a model judge second. A pass needs both layers to agree. If the judge's verdict cannot be read, the dispatcher withholds the answer by default. If the revision budget runs out, it returns the last answer marked as not validated. The precise claim is therefore that every answer carries its verdict and an answer the validator cannot rule on is withheld, not that nothing unvalidated is ever returned.

4.5 MeshGroups: team-scoped access and three admission definitions

A MeshGroup belongs to one organization and binds members to workspaces, model pools and nodes. Each member holds explicit actions in that group (planning inference, reading the workspace,

proposing tool use), and membership in another group is never added in. A Digital Employee's request also requires its human sponsor to be active and to hold the same actions in the same group, so an employee can never reach further than the person accountable for it (R4).

The ThoxWork planner evaluates a routing request in two stages. First, ordered early denials stop requests from the wrong organization, from unknown or revoked principals, without the required actions, without an accountable sponsor, against an unbound workspace or pool, for an unsupported task, or from an unknown source. Second, every node bound to the group and serving the pool is classified by locality (the requester's own device, the same site, another site, or remote) and collects deny reasons: unavailable, model not ready, stale or future-dated observation, data class not permitted, cross-site or remote use not allowed by all three policy levels (organization, group and workspace), and, for non-local nodes, no mesh opt-in, no sharing consent, unverified or unencrypted transport, no ready site gateway, or relay denied. The effective data class is the stricter of the workspace's and the payload's, so a public payload cannot downgrade a restricted workspace. Eligible nodes are ordered device first, then same site, then other sites, then remote, then home site first, then by identifier, and the first is selected. Every response states that it is a preview: no execution, no authorization, no cloud fallback.

Three admission definitions exist in the THOX record, and they answer different questions (Table 5). The product planner chooses by locality and policy and does not consider capacity. The THOX reference admission kit checks tenant, data classification against the group's ceiling, node health and authentication, allowed and authenticated sites, and cross-site use only when policy enables it and the request consents; it then places the whole request on the eligible node with the most free memory, ties to the earlier node, and never compares demand with that node's capacity. The rule of this paper, Algorithm 1, is the third: required resources must be a subset of the group's approved set, demand above the group's pooled capacity for the resource is denied as unplaceable, demand above currently free pooled capacity waits, and there is no partial placement, splitting of the admission decision, or borrowing from another group.

Algorithm 1 · MeshGroup admission (public rule)

```
admit(task, group, free):
 1. Policy:       if task.required is not a subset of group.approved:
                       return DENIED, reason UNAPPROVED_RESOURCE
 2. Placeability: if task.demand > pooled_capacity(group):
                       return DENIED, reason UNPLACEABLE
 3. Current fit:  if task.demand > free:
                       return WAIT        (stays Queued; never placed elsewhere)
 4.               return ADMITTED

There is no partial placement, no demand splitting, no borrowing from
another group, and no fallback group. A DENIED task is terminal and is
shown, with its reason, in the group's Denied tray.
```

Question	Product planner (ThoxWork)	Reference kit	This paper, pooled rule
Unit of decision	one routing preview	one request	one task

Question	Product planner (ThoxWork)	Reference kit	This paper, pooled rule
Who may ask	member with explicit actions; employee needs an accountable sponsor	tenant and group match	task's group
Data limits	stricter of workspace and payload class; node data classes	classification at or below group ceiling	not specified
Sites	cross-site and remote only if organization, group and workspace all allow	allowed, authenticated sites; cross-site needs policy and consent	not specified
Capacity	not considered	not compared with demand	demand against pooled group capacity; wait if not free
Selection	locality, then home site, then identifier	most free memory, then inventory order	admit or deny; placement unspecified
Effect	preview only	returns a node	admits to run

Table 5. The three MeshGroup admission definitions on record, shown side by side rather than merged.

DISPOSITION OF THE ADMISSION QUESTION (D1)

The companion review recommended replacing pooled admission with the planner's scoping plus a single-node capacity check, on the ground that no single request has been run across two nodes. This edition does not adopt that recommendation, because the question was settled by implementation while the edition was in preparation: thoxwork-core implements Algorithm 1 as written, pooled capacity is computed from the group's approved resources and never stored, admission and assignment run in one serializable transaction, and the acceptance replay of Section 7.11 exercised the rule live, including its WAIT path under parallel load (exactly 10 ACCEPTED and 40 WAIT at a saturated pool, with no over-admission). **Pooled admission therefore stands as the governing definition.** The kit remains what it always was: a reference implementation of the placement policy core, valuable as an independent second implementation (Section 7.7 measures agreement with it), not as the admission rule. Two obligations follow and are recorded in Section 13: the planner's preview and the core's admission must never be presented as the same decision; and live dispatch (milestone M3) must honour pooled admission honestly, which means a dispatch shortfall, where admitted work cannot be placed on the day, is a dispatch outcome reported as such, never a retroactive change to an admission verdict. Spanning execution of a single request across nodes is a MeshStack capability question, not an admission question, and this paper claims no such execution.

4.6 Handoffs and evidence

Two evidence mechanisms are built. ThoxEmployee's audit receipts record actor, action, target, rule and outcome before and after each effect, and its security documentation states plainly that the audit table is not a tamper-proof compliance ledger. Each handoff stores the SHA-256 of its body, and accepting a handoff whose body no longer matches is refused. Because the hash lives in the same writable row as the body, this detects accidental change, not an insider who rewrites both (adversary A3). The model therefore specifies a stronger record, which the lab implements and Section 7.9 measures, and which thoxwork-core's append-only event log adopts structurally with hash-chained

entries. A record r_i carries a hash that commits to its content and to the previous record, and, with a session key k held outside the store, an HMAC tag:

$$h_i = \mathrm{SHA256}(\mathrm{canon}(i, t_i, \kappa_i, a_i, \pi_i, h_{i-1})),\ h_{-1} = 0^{256},\ \tau_i = \mathrm{HMAC}_k(h_i) \quad (2)$$

where t_i, κ_i, a_i, π_i are the time, kind, actor and payload and canon is JSON with sorted keys. An anchor is the pair (n, h_{n-1}) of record count and head hash, held by a second party (another MeshGroup member, the sponsor, or a separate store). Handoffs follow the same pattern: each handoff's chained hash commits to the task, the step, the member, the brief, the body hash and the previous handoff.

Proposition 1 (keyed chain). Let a verifier hold k. Any edit of a record's fields, any reorder, any deletion or insertion inside the chain, and any rebuild of the suffix after an edit, made by a party without k, is detected at the first affected position, except with probability bounded by forging HMAC-SHA256. Sketch. An edit at i changes h_i under second-preimage resistance; a rebuild that restores the links must produce τ_i for the new h_i without k.

Proposition 2 (truncation). No check that reads only the stored chain, keyed or not, detects removal of a suffix, because every prefix of a valid chain is itself a valid chain. Detection needs state outside the chain, such as an anchor (n, h_{n-1}) held by a second party. Sketch. Verification of $r_0 \ldots r_{m-1}$ uses only those records and k, and they are unchanged by deleting $r_m \ldots r_{n-1}$.

Section 7.9 confirms both propositions empirically and states the cost. The trace-tampering result of September 2026 adds the deployment rule that follows from them: the workspace, not the agent, must write the record, and the anchor must sit outside the agent's reach [68]. Tree-structured logs would add compact proofs for audits of single events [59, 61]; the design keeps a linear chain because per-team logs are small and verification is cheap at that size.

4.7 Memory that keeps contradictions

ThoxEmployee stores memory in seven tiers. For semantic facts it splits a statement into subject and value at the first colon, equals sign, arrow or form of the verb to be, normalizes case and spacing, and tags a new fact that collides with an earlier fact on the same subject. Both facts are kept and cross-referenced; nothing is merged or dropped (R8). The rule is lexical: a probe during review found that "the build is green" and "build: red" are not tagged because the subjects differ, a false negative stated here as a limit.

4.8 Security argument

R1 and R2 answer excessive agency and tool misuse structurally: an injected instruction (A1) can at most request an action, the gateway decides outside the model, and consequential effects wait for a person [47, 49]. Once the planner authorizes, R4 bounds the blast radius of A1 and A2 to one group, re-checked at each request, consistent with zero trust's rule that access to one resource does not grant another [53]. The sponsor rule prevents an employee from exceeding its accountable person. Stale observations (A4) are rejected by the planner. A5 is not solved by any mechanism here; the record of what the approver saw and when is the mitigation the evidence chain supports, and reviewer capacity

must be planned as a resource. A3 and A6 are what the keyed, anchored chain addresses, with the limit of Proposition 2, and what thoxwork-core's insert-only database grants address structurally: the core's database role cannot update or delete the event log, handoff entries, review events or check-ins, and the separate verifier role cannot insert at all (Section 7.11).

5 What is built: an evidence-bounded account

This section reports the state of the THOX portfolio against the model, from the repository and product-record audit conducted for this paper (method in Section 14) and from the implementation milestone completed on 2026-11-04. Status labels are the paper's own and deliberately conservative: **Built** (code exists and is evidenced), **Live** (built and running as a service, verified against its API), **Preview** (built and deliberately non-executing), **Designed** (specified, partially prototyped), **Proposed** (positioning and plans only). Where THOX's own materials limit a claim, the limit is stated rather than smoothed over.

Component	Status	Evidence of record	Limit carried with the claim
thoxwork-core (server implementation of the model)	Live	A modular monolith in Rust (axum, tokio, sqlx) over PostgreSQL 16, live since 2026-11-04: workspaces, MeshGroups with approved resources and pooled capacity computed (never stored), admission and assignment in one serializable transaction, a server-enforced Rolodex state machine, the binding review gate (Done only through the approve transition), and a hash-chained append-only audit log with insert-only database grants. The S7 verifier is a separate binary on a read-only database role. Acceptance: Section 7.11.	Single-host pilot posture: the service binds a local port behind a private-network HTTPS endpoint; a public TLS step still needs a domain decision; live dispatch to MeshStack and ThoxRoute is milestone M3 and is an unconfigured stub at M1, reported as such by the service itself.
thoxwork-surface (web UI)	Built	The web surface wired to thoxwork-core with authentication on and the earlier silent-fallback routing removed; 338 of 338 surface tests pass.	The surface's OAuth broker secret is operator-provisioned; the gate fails closed until it is set. A server-sent-events endpoint is a stub at M1.
ThoxWork shell (desktop, web, server)	Built, on an upstream base	Built on the open-source OpenWork project, running the OpenCode engine [72, 73]. THOX restyled it and added the governance layers below.	THOX's contributions are the layers, not the shell. Upstream license notices travel with the code (Section 13, D2).
MeshGroups (in the shell)	Preview	Strict schema, host-only preview endpoints, the deterministic planner of Section 4.5, a settings view and an end-to-end browser script.	Execution, grants, persistence, enrollment and multi-node execution are not built in the shell; the executing implementation of the model is thoxwork-core, above. The planner is a preview and never executes (Section 4.5).
Digital Employee gateway	Built	Server-held credentials, closed route list, owner, collaborator and viewer scopes, pinned contract release; the app never holds backend credentials.	Integration into the ThoxWork shell is in development.

Component	Status	Evidence of record	Limit carried with the claim
ThoxEmployee (employee operating layer)	Built (pre-release)	An employee kernel with fail-closed policy (Algorithm 2), an operator action ceiling, digest-bound execution with preview, a persisted audit trail and hashed handoffs; pre-releases workspace-v0.3.0 and migration-studio-v0.2.0. Clean-room from public behavior of OpenBot, OpenMausBot and Rakazo.	Pre-release software. The kernel evidences the bounded-authority half of the model; it is not, by itself, the ThoxWork platform.
Digital Employee contract v2	Built	Assignment API with the seven-state lifecycle of Table 3, approval records, work receipts and event streams.	The gateway's rejection mapping disagrees with the contract (cancelled vs blocked); the contract is the authority (Section 4.3, D4).
Digital Humans framework (Rust) and runtime (Python)	Built	The Rust framework carries the gateway with approval inbox and role-checked decisions, runtime and memory crates. The Python fleet runtime operates the THOX roster and is built on the open-source Hermes Agent [74].	The Python runtime's test suite is largely inherited from its upstream; its counts are reported, not counted, in the census below.
Rolodex directory	Built	Nine-card directory with recorded introductions, consent-gated chat with one employee at a time, and a team selection that is marked for review and grants no tools or permissions.	No ordered task or handoff (Section 4.4). The ordered loop lives in the guest demo engine and, server-side, in thoxwork-core.
ThoxEmployee guest demo	Built (hosted adapter not re-run here)	Ordered team task with written handoffs, revision-fenced steps, human approve or reject, daily limits and 24-hour session expiry, on the ThoxEmployee workspace engine.	Captured in the engine's offline demonstration mode; demonstration outputs are deterministic and labelled as such by the product itself.
RolodexGo (Rolodex client, iOS/macOS)	Built (pre-release)	The shipped client is the 11-file contacts client on the repository main branch (conversation, DigitalHuman profiles, approval inbox, and Sadie Link pairing); the application carries an Xcode Organizer upload receipt (version 1.0.0, build 3). The Teams and Work tabs exist as a combined changeset of 2026-10-01: a Teams tab implementing ThoxEmployee team features (named teams from the DigitalHuman roster, the Queued to In Progress to In Review to Done queue, a written handoff note recorded on every reassignment) and a Work tab implementing the ThoxWork integration catalog (ThoxRoute, ThoxLLM, MeshStack, an MCP tool catalog, and the Sadie Link pairing), with 11 unit tests over the team models. The changeset was built and is test-covered, but it never landed in the repository main branch or in the build tree; it is preserved as an unlanded changeset, and whether the uploaded 1.0.0(3) build included it is unverified, so this paper does not claim that it did.	Local-first client. The claim for the shipped client is the contacts client plus its upload receipt, not live remote enterprise execution; the native preview is unpaired. In the changeset, team and work state are held on device and connection states in the Work tab are honest by construction, with integrations reporting unconfigured until configured. The tabs land in the shipped client as part of milestone M2.

Component	Status	Evidence of record	Limit carried with the claim
Validated dispatch	Built	Guardrails, planning, parallel workers, synthesis and the two-layer validator of Section 4.4; unreadable verdicts withheld, budget exhaustion marked not validated.	Every answer carries its verdict; the claim is not that nothing unvalidated is ever returned (Section 4.4).
MeshGroup reference admission kit	Built (reference)	The reference admission policy of Table 5 with its own tests (8 of 8), including cross-site default-deny and consent-gated placement scenarios; a model-configuration kit publishes enterprise workgroup configurations on upstream weights.	Not the admission rule (Section 4.5, D1) and not wired to the ThoxWork planner. The kit's models are published configurations on upstream weights and are explicitly not THOX-trained; group isolation in those configurations is a prompting and routing convention.
MeshStack	Built and published	Research paper v1.0.1, published with a reproducibility artifact [2]; backend test suites recorded at 241 of 241 and 25 of 25 smoke tests in THOX records.	Cited, not re-disclosed: this paper relies on the published MeshStack record and does not restate its internals. The corrected KV-compression range from the MeshStack v1.1.0 evidence work (1.17 to 1.20x on real GGUF workloads) supersedes the earlier 1.51x figure, which no THOX surface should use.
ThoxWork platform (purchasable enterprise product)	Proposed, in active development	The model is implemented server-side by thoxwork-core (Live, above); a production web surface, a desktop workspace and the ThoxWork Creative Suite v0.1.0 (seven Rust applications, a zero-dependency core, 22 tests, real demo outputs preserved as evidence) exist as related workstreams.	THOX's canonical product copy describes the purchasable enterprise platform as Coming Soon, and this paper's platform claims are claims about the model, its lab evaluation and the pilot implementation, scoped exactly to that.
DigitalHuman roster and agent fleet	Built	A nine-role DigitalHuman roster operates in THOX's own work today under the Colleague and Disclosure Standard [75], and the ThoxAgent fleet (v1.2.0 line) evidences deterministic allocation and queued-not-dropped dispatch in an independent code audit (9 of 9 tests).	Internal operation evidences the roster and roles in THOX's own work. It is not a customer deployment claim, and none is made.

Table 6. Component status against the ThoxWork model. Status labels are the paper's own, assigned from the audit and milestone evidence, and are deliberately conservative.

5.1 Test census

Each repository's own suites were rerun for this paper, one at a time, in isolated copies, and counts were read from each tool's own summary output. The 18 rerun suites passed **3,747** tests and failed **3**, with **34** skipped. Two failures are in one ThoxWork server suite and the third is in the Rolodex

directory; all three are tracked internally. A further suite, the Python agent runtime's, was parsed from the review team's run of the same commit rather than rerun (24,109 passed, 72 failed, 52 errors); it is largely the Hermes Agent suite the runtime inherits, so it says more about the upstream than about THOX's additions, and it is reported here, not counted in the headline. Seven suites did not run because they need a macOS host, a container runtime, a disposable database, hosted credentials or a live model provider. Browser end-to-end assertion groups (17 passed) and 10 scripted lint, type, format and project checks (all passed) are reported separately; a contract check that does not yet gate on its output is not counted. The Evidence Lab's own 75 unit tests pass and are excluded from the product totals.

Component	Commit	Suites run / log / not run	Passed	Failed	Skipped	Notes
MeshGroup reference admission kit	657c7c4	1 / 0 / 0	8	0	0	Kit only
ThoxWork shell and servers	ab29b61	18 / 0 / 2	975	2	0	2 failures in one server suite, tracked internally; includes tests inherited from OpenWork; hosted enterprise and live-provider suites not run
ThoxEmployee runtime	efe63da	4 / 0 / 2	752	0	6	Swift kit and Docker kernel suites not run (no macOS host, no container runtime)
Digital Employee contract service	8997830	2 / 0 / 1	510	0	28	PostgreSQL integration suite not run (no disposable database); 28 skipped for the same reason
Digital Humans gateway (Rust)	73088ef	3 / 0 / 0	337	0	0	Rust workspace tests; clippy with warnings as errors and rustfmt also pass
Digital Humans agent runtime (Python)	c7b4c33	0 / 1 / 0	24,109	72	160 (+52 errors)	Parsed from the review team's run, not rerun; suite largely inherited from Hermes Agent; failures and errors concentrated in browser-automation, SSH, chat-platform and terminal tests that need components the review host lacked
Rolodex directory	eeef7de	3 / 0 / 1	481	1	0	1 failure, tracked internally; browser suite replaced by captures
Thoxy dispatch	2ee63c6	2 / 0 / 0	257	0	0	Backend and web suites
thox.ai website	8999c14	1 / 0 / 1	135	0	0	Claim-related subset only; hosted guest-demo check needs service credentials and was not run
RolodexGo	cdee70c	1 / 0 / 0	292	0	0	TypeScript and domain suites; Swift targets not compiled, so this count is not shipped-client coverage

Table 7. Product test suites, rerun for this paper on one host (2 vCPU Intel Xeon at 2.10 GHz, 8 GB RAM, Linux, Node v22.23.4). Counts are read from each tool's own summary output; subset and repeat runs are excluded.

5.2 Lineage

The ThoxWork shell is built on the open-source OpenWork project and runs the OpenCode engine [72, 73]; the Digital Humans Python runtime is built on the open-source Hermes Agent [74]; ThoxEmployee is a clean-room implementation from the public behavior of OpenBot, OpenMausBot and Rakazo. THOX's contributions are the governance layers built on those foundations: the gateway, the approval gate and contract, the MeshGroup planner and admission rule, the Rolodex surfaces, and thoxwork-core. Upstream license notices travel with every distribution; Section 13 (D2) records the standing decision on credit. The first edition did not state this lineage; Appendix C records the correction.

Two audit findings deserve plain statement because they bound everything that follows. First, the strongest single integration evidence in the client estate remains the RolodexGo combined changeset of 2026-10-01: it is the one artifact in which ThoxEmployee team behavior and ThoxWork integrations exist together in one client codebase, implementing exactly the workflow of Figure 2 and the handoff rule of Definition 3. It is a built, test-covered changeset that never landed in the shipped client, and it is claimed here as exactly that, no more; milestone M2 lands it. Second, there is one named gap: voice calling for DigitalHumans (ThoxCall) has no repository or work product in the audit scope and is deprioritized in THOX planning; it appears in this paper only in this sentence, as a roadmap item with no evidence behind it.

6 Method: three instruments

The evaluation uses three instruments, one per evidence tier, and the paper never mixes their numbers. The **oracle** (Section 6.1) tests the model's rules as specified. The **Evidence Lab** (Section 6.2) tests whether independent re-implementations of the product's rules agree with the product's own code. The **live harness** (Section 6.3) tests whether the running implementation enforces the rules the oracle specified, using the oracle's own test generators. Where any two instruments differ on the same question, the disagreement is reported, not averaged.

6.1 The protocol oracle (SIMULATED)

The oracle is a deterministic simulator (`thoxwork_experiments.py`, Python standard library only) implementing Definitions 1 to 5 and Algorithm 1 exactly, with an independent checker that reads only the final event log and task records and never calls the admission or workflow code it verifies. The THOX Virtual Lab browser artifact is a view and manual driver of the same model: roster and MeshGroup panels showing pooled capacity computed by the Definition 1 formula, the Rolodex board with its four columns and a visible Denied tray, guards that reject illegal actions with the failed guard named, and experiment runners that display the oracle's summary values under the evidence notice. Where browser and oracle ever differ on the same seed, the oracle wins by definition.

Determinism. Master seed 20261104; one random generator per experiment and replicate, seeded by a fixed formula; integer ticks; sorted iteration. There are no timestamps in the evidence file, and the suite's self-test re-runs experiments and asserts byte-identical serialized results. Two full independent

runs of the suite produced the same summary file, byte for byte (Section 14), and the file was re-verified byte-identical again for this edition.

Pre-registration. The pass bars in Table 8 were fixed in the lab protocol dated 2026-11-04, before the first run. A falsified bar would have been reported as falsified; the bars were not adjusted after any result was seen.

Exp.	Invariant	Pre-registered bar
E1	I1 policy	Zero leaks in every replicate; aggregate false-block rate at most 0.01; audit coverage 1.0
E2	I2 handoffs	Chain completeness 1.0; note coverage 1.0; zero probe leaks in every replicate
E3	I3 admission	Silent overflow 0; cross-group spills 0; UNPLACEABLE recall 1.0; stranded 0; right-sized placement at least 0.99
E4	I4 review gate	Zero bypass successes in every class; zero Done-without-review; rejection integrity 1.0; legitimate completion at least 0.99
E5	I5 scale	Every grid cell completes at least 0.99 of its tasks; scaling efficiency at team size 16 at least 0.50

Table 8. Pre-registered pass bars, fixed in the lab protocol before any run.

Simulated-latency model (E5 only). Time is an abstract tick; a tick is not a millisecond of any real system. Service time per task is 10 + 2 per required resource + a uniform 0 to 4 ticks, drawn once at creation; review occupancy is 3 ticks; a coordination term of floor(0.5 × (G − 1)) ticks is added per task, where G is the MeshGroup count. Workers hold at most 2 concurrent tasks; a group's reviewer holds at most 4 concurrent reviews. Arrivals are a batch at tick 0. Reviewers in E5 are cooperative by design, because E5 measures throughput, not the gate; the gate is E4's claim. The E5 runner is event-driven; its semantics are the tick semantics above with idle time compressed, which cannot change any task's start, finish or ordering.

Workloads. E1 runs 1,000 attempts per replicate (400 legitimate; 250 direct violations; 200 cross-group; 150 post-admission escalations). E2 runs 300 tasks per replicate with reassignment depths 0 to 5, plus 200 negative probes. E3 runs 500 tasks per replicate over a demand sweep from 0.10x to 2.00x of a 120-unit pooled capacity, on a group drained to empty before each task so that queueing never confounds placeability. E4 runs 200 legitimate tasks, 300 bypass attempts in six classes, and a 200-task control arm on an otherwise identical group with the review gate switched off. E5 runs a 4 × 4 grid (team sizes 2, 4, 8, 16; MeshGroup counts 1, 2, 4, 8) of 2,000 tasks per cell. Replicates: 20 for E1 to E4, 10 per cell for E5. Resources and capacities in the lab are abstract units (RU) over eight synthetic resource types; no mapping to real compute, storage or cost is claimed. Rosters are synthetic. The lab contains no product internals, no customer data and no real model or device specifications.

6.2 The Evidence Lab (MEASURED, code and lab)

The Evidence Lab is a single HTML file that runs in any modern browser, opens offline, never calls a model and never takes an external action; every employee output is deterministic demonstration text, and the example organization is fictional. All logic is in four dependency-free modules shared with the experiment scripts, so the page and the measurements run the same code. Its ledger is a keyed chain of Equation 2; the session key is generated in the browser and never leaves the tab, an exported

bundle carries hashes and tags but not the key, and an imported bundle can be checked for hash links by anyone. The page runs SHA-256 and HMAC known-answer tests on load (FIPS 180-4 examples and RFC 4231 cases).

Bench	What it runs	Grounding
MeshGroup admission	The reference kit rule with per-node reasons, and Algorithm 1 beside it on the same request	Agreement with the kit, Section 7.7
Routing planner	A clean re-implementation of the ThoxWork planner rules	Agreement with the product function, Section 7.6
Approvals	Algorithm 2 gateway, the Equation 1 approval gate with role check, and the Table 3 lifecycle	Rules read from the ThoxEmployee kernel and the v2 contract; unit tests
Ordered handoffs	A team runs a brief in order; body hashes as built and chained hashes as specified	Unit tests; Proposition 1
Tamper tests	The attack classes of Section 7.9 on a copy of the live ledger, checked by hash links, by the session key and by an anchor	Experiment E9, Section 7.9
Captures	Screenshots, screen recordings and PDFs hashed in the browser and written into the ledger; a self-rendered snapshot of the ledger head	SHA-256 known-answer tests on load

Table 9. Evidence Lab benches.

One framing fact about the lab is recorded here because it shaped the companion review: the lab's admission bench leads with the reference kit's placement rule, because the kit is the implementation the bench was built to check. That is a property of the bench, not of the model. In this paper the admission rule is Algorithm 1 everywhere, the kit is a comparator, and Section 7.8 measures the two under the paper's semantics. The interactive demo published with this paper implements Algorithm 1 semantics only.

6.3 The live replay harness (MEASURED, live)

For E11 the oracle's E1 to E4 generators were ported one-to-one to a harness that drives the live thoxwork-core API instead of the simulator: same experiment structure, same seed formula (seed(E, rep) = 20261104 + 10000·E + rep), same replicate count (R = 20), same pre-registered bars. Three judges score the outcome independently: the S7 verifier through its API, the S7 verifier through its CLI on a read-only database role, and a checker inside the harness itself. The run took approximately 104 minutes of wall time against the deployed service. Four method adaptations are stated plainly: creation-time admission in the live system is protocol-governed (a workspace is created within its group's bounds and Algorithm 1 is evaluated at assignment, where the harness exercises it); the WAIT drain path is not exercised by the sequential drivers, so WAIT is evidenced by the dedicated parallel test instead; E2 tasks are submitted to In Review to free the pool for subsequent tasks; and one defect the drills found, in credential scoping, was fixed and the affected drill re-run in full before the acceptance numbers were taken (Section 7.11).

Hosts. The code and lab measurements of Sections 7.6, 7.7 and 7.10 ran on one host (2 vCPU Intel Xeon at 2.10 GHz, 8 GB RAM, Linux, Node v22.23.4); timings from it are single-host measurements and

are labelled as such. E8 and E9 ran for this edition on the build host under the seed discipline of Appendix A. E11 ran against thoxwork-core deployed on a single Azure host with PostgreSQL 16; it is a pilot deployment, and no figure from it is a production benchmark.

7 Evaluation

Sections 7.1 to 7.5 report the oracle (SIMULATED; every number read from `exp/results/summary.json`, the suite's single evidence source; derived ratios are CALCULATED from simulated counts). Sections 7.6 to 7.10 report the code and lab measurements (MEASURED). Section 7.11 reports the live replay (MEASURED, live). The tiers are labelled in every heading so no number can be quoted out of its tier.

7.1 E1 · Policy enforcement (SIMULATED)

Across 20 replicates, 12,000 violating attempts produced **0 leaks**: no direct, cross-group or escalation attempt was ever observed running with an unapproved resource attached. The block rate is 1.0. Of 8,000 legitimate attempts, **0 were falsely blocked** (false-block rate 0.0, against a bar of 0.01). Every attempt, accepted or rejected, appears in the event log: audit coverage 1.0. The independent checker confirms the invariant over the full log: no task ever held an unapproved resource in any running state. Bar: **met**.

7.2 E2 · Handoff integrity (SIMULATED)

Across 20 replicates, 5,000 tasks were reassigned, accumulating 15,000 handoff-chain entries to a maximum depth of 5. Chain completeness is **1.0** and note coverage is **1.0**: every entry is in sequence, correctly linked, and carries a note whose hash recomputes. The 4,000 negative probes (empty notes, whitespace-only notes, reassignment to a non-member, reassignment attempted from In Review) produced **0 leaks**: every probe was rejected and no probe mutated any chain. Bar: **met**.

7.3 E3 · Admission honesty (SIMULATED)

Across 20 replicates, all **4,000 oversized tasks were correctly denied** UNPLACEABLE (recall 1.0), with 0 silently placed, 0 cross-group spills and 0 stranded. All 6,000 right-sized tasks were placed (placement rate 1.0). Denial is issued in the same tick the task is evaluated (mean and maximum deny latency 0 ticks; see Section 9 on what that number means). Figure 3 shows the boundary exactly: at a demand of 1.00x pooled capacity, all 50 tasks per replicate are admitted; at 1.01x, all 50 are denied.

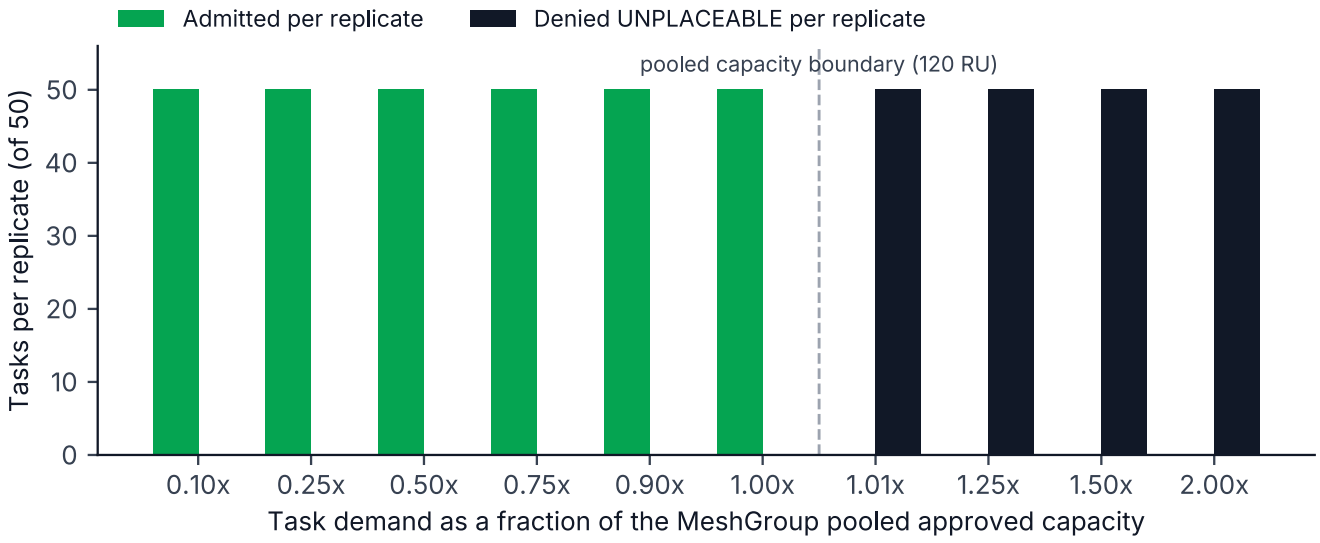


Figure 3. E3 demand sweep against a pooled approved capacity of 120 RU (per-replicate counts, identical across all 20 replicates). The step at the boundary is the Definition 5 placeability rule operating as specified. Evidence class: SIMULATED.

7.4 E4 · The review gate (SIMULATED)

Across 20 replicates, **0 of 6,000 bypass attempts succeeded**, in every class: direct finish (B1), skip review (B2), self-review (B3), wrong-role review (B4), non-member review (B5) and stale approval (B6) each record 1,000 attempts and 0 successes. Every rejected bypass left the task's state, assignee, handoff chain and review events byte-identical (rejection integrity 1.0). All 4,000 legitimate tasks completed through the full assign, check-in, submit, approve path (legitimate completion rate 1.0), and the checker finds 0 Done tasks without a legitimate review. The control arm matters as much as the result: on the identical model with the review gate switched off, 4,000 of 4,000 tasks completed without review. The gate, and nothing else in the model, is what produces the invariant, and the checker demonstrably distinguishes the two regimes. Bar: **met**.

7.5 E5 · Scale and throughput (SIMULATED)

All 16 grid cells completed **1.0** of their 2,000 tasks within budget. Throughput (tasks per 1,000 simulated ticks, cell means) ranges from 124.9 in the smallest cell (2 members, 1 group) to 9,551.2 in the largest (16 members, 8 groups). Group scaling at team size 8 is close to linear in group count (ratios 1.99, 3.70 and 6.46 at 2, 4 and 8 groups against the single-group baseline), which is the expected behavior when groups are independent pools: adding a MeshGroup adds a full team and a full resource pool, it does not subdivide one. The reviewer bottleneck is visible where the model says it should be: the share of completion time spent waiting for review is near zero at team sizes up to 4 and rises to roughly 0.20 to 0.30 at team size 16, where a single reviewer per group becomes the binding constraint. Scaling efficiency is reported and then discounted in Section 9. Bar: **met**, on the non-collapse criterion the bar actually specifies.

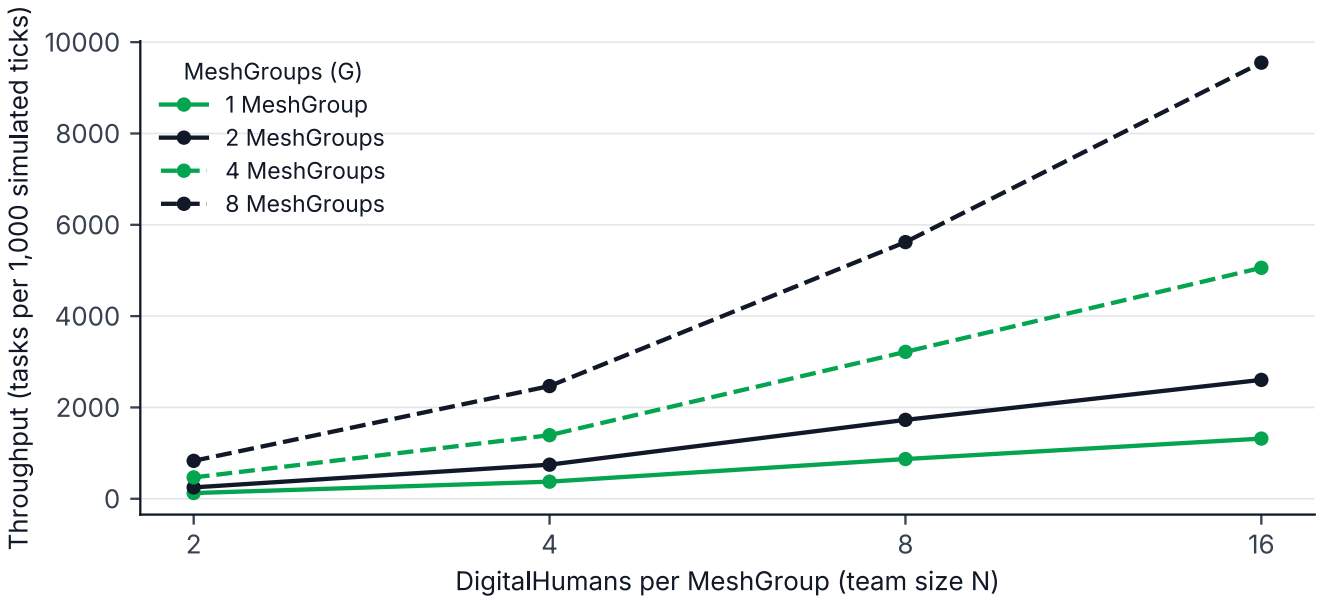


Figure 4. E5 throughput by team size and MeshGroup count (cell means over 10 replicates). Ticks are abstract simulation units. Evidence class: SIMULATED; ratios CALCULATED from simulated counts.

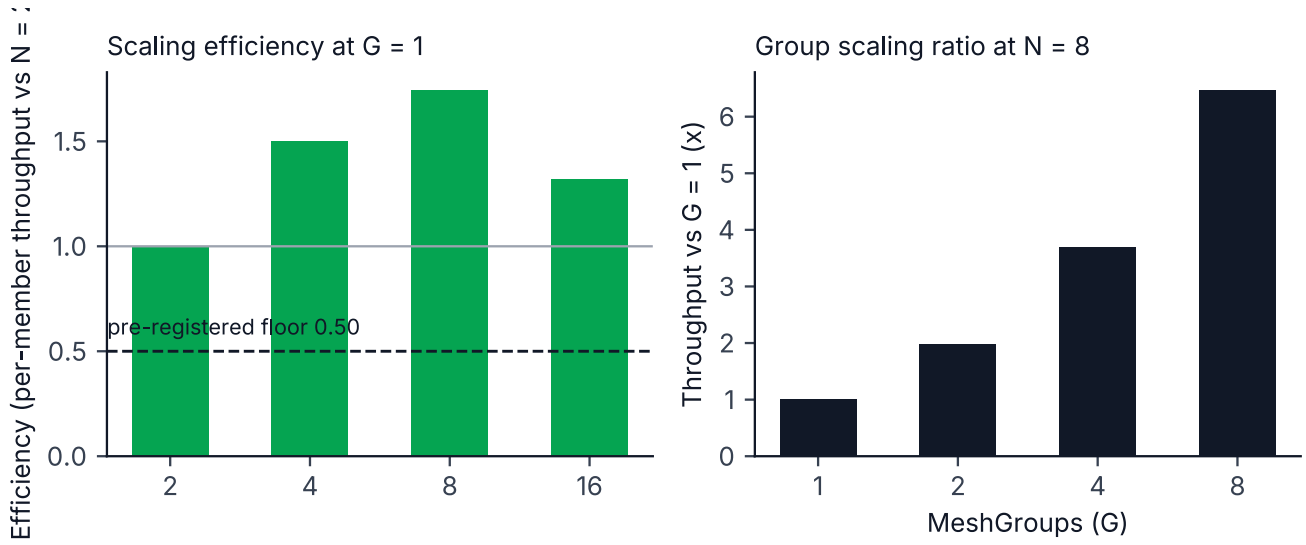


Figure 5. Left: scaling efficiency at one MeshGroup (per-member throughput relative to the 2-member baseline), with the pre-registered 0.50 floor. Right: group scaling ratio at team size 8. Values above 1.0 on the left are a model artifact analyzed in Section 9, not a speedup claim. Evidence class: SIMULATED / CALCULATED.

7.6 E6 · Planner agreement (MEASURED, code)

The lab's routing planner is a clean re-implementation of the ThoxWork planner's documented rules. On 250 seeded MeshGroup directories (1 to 100 nodes, up to 6 groups each) and 12,000 routing requests mixing allowed, denied and unservable cases, every input passing the product's own schema, the lab planner and the product planner (imported from the ThoxWork source at commit ab29b61) agreed on status, reason, selected node and every candidate's deny-reason set for **12,000 of 12,000** requests, and on every compared field, including candidate order, locality, effective data class and the order of deny reasons, for 12,000; **55,832 of 55,832** per-node verdicts matched. The

outcome mix was 3,038 eligible, 5,127 with no eligible node and 3,835 denied before candidates were listed, covering all 9 early denials and all 11 per-node deny reasons. On 4,000 deliberately altered inputs of 18 kinds, the product schema and the lab validator gave the same accept or reject verdict 4,000 times. Agreement shows that the lab runs the product's rules as implemented; as Section 12 notes, it does not show the rules are right.

7.7 E7 · Kit agreement (MEASURED, code)

The lab's reference-rule module and THOX's reference admission kit (commit 657c7c4) decided 20,000 seeded requests (200 scenarios of 50 nodes and 100 requests, with ties in free memory made common on purpose). They agreed on **20,000 of 20,000**, including every tie. The outcome mix was 4,570 placed, 4,377 admitted with no eligible node, and 11,053 denied before placement (unknown group 932; tenant mismatch 1,024; classification above the group ceiling 9,097). This agreement is the strong kind: the kit is a second, independent implementation, not a re-implementation of the lab module. What the agreement establishes is that the lab's kit bench measures the kit's rule faithfully. It does not make the kit's rule the admission rule; Section 4.5 dispositions that question, and Section 7.8 measures what the kit's rule does with capacity, which is the property that matters.

7.8 E8 · Admission divergence under Algorithm 1 semantics (MEASURED, lab)

Why this run exists. The companion review compared the kit's placement rule with a port of Algorithm 1 and reported a wide divergence. The port deviates from Algorithm 1 in three ways, each visible in its delivered source: the kit's request-level denials are pre-applied to the pooled arm, so the comparison does not isolate placement; the pooled sum runs over free memory in GB on every node, not over the Definition 1 resource units of the devices that run the requested resource (Definition 5 membership); and the harness holds nothing in use, so the WAIT branch never fires. E8 repeats the study under this paper's semantics, in two phases, and quantifies each deviation. Arm P is Algorithm 1 (oracle verdict order, node-refined per Definitions 1 and 5). Arm S is the strict single-node capacity rule: admit only if some device running the resource has free capacity at least the demand. Arm X re-implements the port's semantics for deviation accounting only: membership-blind free sums, no WAIT branch. The approved-set gate is identical in all arms, isolating deviations two and three; deviation one is documented from the delivered port source, not re-run.

Phase 1, probe. 20,000 stateless (snapshot, request) probes (25 replicates × 800; worlds of 12 devices with capacities 60, 120 or 240 RU, each approved model hosted by at least two devices; per-device use drawn up to 95 percent; demands from the stated mixture in Appendix A, 12 percent of requests asking an unapproved resource). Algorithm 1 admitted 15,982, waited 1,212, denied 523 UNPLACEABLE and 2,283 for an unapproved resource. Of its admissions, 12,100 fit a single device and **3,882 required the pool**: no single device running the resource had free capacity at least the demand, so the strict single-node rule refuses all 3,882 (19.4 percent of probes; 24.3 percent of pooled admissions). The converse never occurs: the single-node rule admitted nothing Algorithm 1 refused (0 cases), which is the expected subset relation. The port semantics over-admit: Arm X admitted 17,138, which is 1,156 more than Algorithm 1, and the excess decomposes exactly into 967 admissions made against free capacity on devices that do not run the requested resource (phantom capacity) and 189

admissions of requests whose demand exceeds the pool's entire capacity for the resource, which Algorithm 1 denies UNPLACEABLE. The port's missing WAIT branch cuts both ways in the same run: of Algorithm 1's 1,212 WAIT verdicts, Arm X admits 967 (the phantom set) and flattens the remaining 245 into refusals. Three-way verdict agreement across the arms is 14,962 of 20,000.

Phase 1 count (of 20,000 probes)	Arm P (Algorithm 1)	Arm S (single-node)	Arm X (port semantics)
Admitted	15,982	12,100	17,138
of which a single device fits	12,100	12,100	n/a
of which spanning is required	3,882	0	n/a
Wait (queued, not refused)	1,212	no queue	no WAIT branch
Denied UNPLACEABLE	523	(refused)	(refused)
Denied, unapproved resource	2,283	2,283	2,283
Refused, other	0	5,617	579

Table 10. E8 Phase 1. Cross-arm accounting: pooled-only admissions 3,882 (all spanning); single-node admissions refused by Algorithm 1: 0; Arm X phantom admissions 967; Arm X admissions above pool capacity 189; Arm X flattening of WAIT verdicts 245 refusals and 967 admissions. Shares depend on the stated demand model; the subset relations and the deviation decomposition do not.

Phase 2, stream. A sequential arrival stream (25 replicates × 400 arrivals, plus a drain phase) under Algorithm 1, with completions freeing capacity and the WAIT queue re-evaluated each tick, exercises the path the port never reaches: of 10,000 arrivals, 3,311 were admitted immediately, **5,093 waited and were then admitted** as capacity freed, 74 were still waiting at the end of the drain window, 306 were denied UNPLACEABLE and 1,216 denied for an unapproved resource. Of the 8,404 admissions, 4,625 (55.0 percent) consumed capacity on more than one device. The safety invariants were asserted on every tick of every replicate: no device was ever consumed beyond its capacity, and accounted in-use capacity equalled consumed capacity throughout: **0 violations**. On independent state over the same stream, the single-node rule admitted 1,266 and refused 8,734, including 2,244 requests Algorithm 1 admitted; the arms' states diverge once verdicts differ, so this contrast is distributional context, not a same-state counterfactual.

The port's own numbers, in their place. For completeness, the companion port study was re-executed from its delivered artifacts for this edition and reproduced exactly (Appendix A): of the kit's 4,570 placements, 2,572 (56.3 percent) land on a node with less free memory than the demand, because the kit never compares demand with capacity; and the port credits pooled admission with 4,333 admissions the kit cannot serve (1,655 spanning, 2,678 cross-site). Those figures are properties of the kit and of the port, stated here as such. They are not measurements of Algorithm 1, and no other section of this paper uses them as such. What survives translation into the paper's semantics is the qualitative finding, now measured properly in Phase 1: the definitions genuinely disagree, pooled admission serves a large class of requests no single node can serve, and a capacity-blind placement rule places work on nodes too small for it.

7.9 E9 · Audit-chain detection matrix (MEASURED, lab)

Equation 2 was re-implemented from the equation alone for this edition (canonical JSON encoding stated in Appendix A; a fixed test key, not a production key) and subjected to eight attacks in 500 seeded trials each, on chains of 1,000 records, under four verification configurations: hash links alone; links plus an external anchor; links plus the HMAC key; and both. The honest chain passes every check in every configuration, asserted on every trial. Table 11 reports detections out of 500.

Attack	Hash links	+ Anchor	+ HMAC key	+ Key and anchor
Modify a record's payload	500	500	500	500
Delete an interior record	500	500	500	500
Swap two adjacent records	500	500	500	500
Renumber every record	500	500	500	500
Edit a record and rebuild every later hash	0	500	500	500
Truncate the last 50 records	0	500	0	500
Append a forged record, attacker without the key	0	500	500	500
Append a forged record, attacker with the key	0	500	0	500

Table 11. E9 detection matrix: trials detected of 500, 1,000-record chains. The rebuild row was produced by the supplement run of Appendix A under the same builder, checks, seeds and trial count.

The matrix confirms both propositions empirically, cell for cell. Interior tampering (modify, delete, swap, renumber) is detected by every configuration, hash links included: the built mechanisms of Section 4.6 detect accident and casual tampering, as stated. The rebuild row is the insider boundary (A3): an attacker who can recompute hashes defeats hash links in all 500 trials, and is caught in all 500 by the key they do not hold (Proposition 1) and by the anchor, because the rebuilt suffix changes the head hash. Truncation is the structural limit (Proposition 2): no in-band check, keyed or not, detects it in any trial (0 of 500 in both unanchored columns), and the external anchor detects it in all 500. The append rows mark the key's limit: an honestly linked forged record is indistinguishable from a real one to hash links (0 of 500); without the key its forged tag is caught in all 500 trials; with the key, tags verify and only the anchor, which binds the record count and the head, still detects the append in all 500. The deployment conclusions are the ones Section 4.6 states: the record must be written by the workspace rather than the agent, the key must live outside the store, and the anchor must sit with a second party. The companion edition's matrix, measured on its JavaScript implementation, agrees qualitatively in every cell.

Cost on this edition's build host (Python standard library; medians; host measurements, not part of the reproducibility claim): a keyed append costs a median of 10.0 microseconds per record and keyed verification 10.5 microseconds per record on a 1,000-record chain; verifying a 10,000-record keyed chain takes a median of 197 ms; an exported record costs about 228 bytes. The companion implementation (pure JavaScript SHA-256) measured 27.6 microseconds per keyed append and 307 ms for the same 10,000-record verification on its host; the two measure different implementations on different hosts and are reported side by side only to show the cost class: at team scale, keyed and

anchored evidence costs microseconds per record. Known-answer tests for the implementation: 7 of 7 (FIPS 180-4 SHA-256 examples; RFC 4231 HMAC cases 1, 2 and 6).

7.10 E10 · Decision latency (MEASURED, single host)

Admission and planning decisions are cheap at the scale of a team. The lab module took a median 0.11, 1.05 and 13.3 microseconds per decision for inventories of 10, 100 and 1,000 nodes, and the THOX reference kit in Python 1.57, 6.3 and 50.1 microseconds; cost grows with inventory size because every node is checked, and batches include early denials, which cost almost nothing, in the proportions of Section 7.7. Planning costs more than admission, and validation costs more than planning: with 100 bound nodes, a full product preview took a median 783 microseconds, of which parsing the request schema alone took 608 (about 78 percent; at these sizes the ratio moves by a few points between runs), while the lab's plan step alone took 83 microseconds. Validating a full 100-group directory took 42.3 ms with the product schema and 6.7 ms with the lab's validator. All figures are medians of 30 interleaved repetitions on the single host of Section 6.3 while it was otherwise idle; they are decision costs on that host, not device or production benchmarks.

7.11 E11 · The live replay (MEASURED, live)

The centre of gravity of this edition is a simple experiment: take the generators that produced the simulated E1 to E4 results, point them at the running implementation instead of the simulator, and see whether the bars survive contact with real code. The implementation is thoxwork-core (Section 5), a Rust modular monolith over PostgreSQL, with admission and assignment in one serializable transaction, pooled capacity always computed and never stored, the review gate enforced by the state machine (Done is reachable only through the approve transition), and database grants that make the audit tables insert-only for the core's own role. The verifier, S7, is a separate binary running on a read-only database role, so the component that checks the record cannot write it. Acceptance run tag `f182809`; approximately 104 minutes of wall time; pre-registered bars unchanged from Table 8.

Test	Oracle result (SIMULATED)	Live result (E11)	Bar
E1 policy	0 / 12,000 leaks; 0 / 8,000 falsely blocked; audit coverage 1.0	0 / 12,000 leaks; 0 / 8,000 falsely blocked; audit coverage 1.0 in every replicate	met
E2 handoffs	Completeness 1.0; note coverage 1.0; 0 / 4,000 probe leaks	Completeness 1.0; note coverage 1.0; 5,000 tasks, 15,000 entries, max depth 5; 0 / 4,000 probe leaks	met
E3 admission	4,000 / 4,000 oversized denied; 6,000 / 6,000 placed; boundary step at 1.00x / 1.01x	4,000 / 4,000 oversized denied; 6,000 / 6,000 right-sized placed; 0 silent, 0 spills, 0 stranded; boundary: demand 120 RU admitted, 121 RU UNPLACEABLE, in every replicate	met
E4 review gate	0 / 6,000 bypasses; 4,000 / 4,000 legitimate completions; control 4,000 / 4,000 without review	0 / 6,000 bypasses (B1 to B6, 1,000 each); 4,000 / 4,000 legitimate completions; 0 Done-without-review over 8,000 Done tasks; rejection integrity 1.0; control arm (gate off) 4,000 / 4,000 completed without review	met
Parallel admission	(oracle is sequential; not tested)	50 parallel admissions of 12 RU against a 120 RU pool: exactly 10 ACCEPTED, 40 WAIT, 0 DENIED, 0 REJECTED; sum of in-progress demand exactly 120 RU; no over-admission	met

Test	Oracle result (SIMULATED)	Live result (E11)	Bar
Three judges	(oracle checker only)	S7 verifier API, S7 verifier CLI and the harness checker agree on all 102 workspaces; a fleet sweep of the S7 verifier over 121 workspaces reports 0 non-clean	met

Table 12. E11 live replay against thoxwork-core, acceptance run f182809. The oracle column is reproduced from summary.json for comparison; the live column is measured on the deployed service.

Three findings from the replay matter beyond the pass marks. **First, the harness found a real defect.** The suspend and revoke drills failed for principals whose identifiers were shared across workspaces, because the credential routes resolved principals without workspace scoping. The fix scopes those routes by workspace and adds a token-epoch migration, so rotation, suspension and revocation invalidate outstanding tokens immediately; the affected drill was then re-run in full and passed. A replay that finds nothing would be worth less; this one found, and fixed, exactly the class of cross-workspace defect I1 exists to prevent, one layer down. **Second, the structural claims were tested, not asserted.** The insert-only posture was proven against the live database: update and delete on the event log, handoff entries, review events and check-ins are denied to the core's role, and insert is denied to the verifier's role. The web surface wired to the core runs with authentication on and its previous silent fallback removed, so a failed core call surfaces as an error rather than as plausible local data. **Third, the limits are part of the record.** Dispatch from admission to MeshStack or ThoxRoute execution is an unconfigured stub at this milestone and reports itself as such; autonomy tiers are carried and displayed but tier-based action-class gating is not yet enforced; per-principal HMAC secrets with short-lived tokens stand in for the asymmetric host keypairs of the fuller design; and the write path is serialized on a single connection as a pilot posture. Section 13 turns each of these into a milestone rather than leaving them as caveats.

7.12 Product evidence captured (MEASURED)

The review captured the real running products, never mockups, at 1280 by 800 and 390 by 844 pixels, and recorded screen videos of complete flows: the ThoxWork MeshGroups routing preview denying a restricted workspace; the ThoxEmployee workspace holding output for a person's review; the ThoxEmployee kernel's fail-closed policy and a hashed handoff; and the guest-demo engine's ordered team handoffs before review. A subset reviewed against the disclosure boundary of Appendix D ships in the reproducibility package with SHA-256 hashes; captures that show local ports, device registries or generated identifiers are withheld. The native clients were captured separately in the implementation milestone's own record; the RolodexGo status claim of Section 5 does not rest on captures.

8 Governance mapping

Table 13 maps the mechanisms of ThoxWork and of the Digital Employee stack beneath it (ThoxEmployee, the Digital Employee contract and the Digital Humans gateway) to the oversight, record-keeping and access requirements of Section 2.4. It states alignment with the text of each

document, not conformance; no audit has been performed, the NIST framework is voluntary, and whether the EU AI Act binds a deployment depends on its use.

Requirement	Source	Mechanism	Status
Define roles for human-AI configurations and oversight	NIST AI 100-1 GOVERN 3.2 [42]	Named Digital Employees with roles and accountable human sponsors; each approval request names its required role	Built in the Digital Employee stack
Document oversight processes	NIST AI 100-1 MAP 3.5 [42]	Approval reasons, required role and decision recorded per request	Built in the Digital Employee stack
Content provenance	NIST AI 600-1 [43]	Hashed handoffs and receipts as built; keyed, anchored chain as specified in Equation 2	Built in part (ThoxEmployee); specified for the platform record
Automatic recording of events	EU AI Act Art. 12 [45]	Audit receipts around each gated effect; lifecycle events per assignment; insert-only event log in thoxwork-core	Built in ThoxEmployee; built in thoxwork-core
Override, reverse or stop	EU AI Act Art. 14 [45]	Person-only approval; reject blocks; cancel is terminal; previews never execute	Built in the Digital Employee stack
Keep logs at least six months	EU AI Act Art. 26(6) [45]	Local-first records owned by the organization; retention policy is the deployer's	Deployer duty
Management system for AI	ISO/IEC 42001 [44]	Evidence the system can supply to an organization's management system	Supporting evidence only
Excessive agency, complete mediation	OWASP LLM06 [47]	Fail-closed gateway outside the model; operator ceiling; approval for effects	Built in ThoxEmployee
Identity and privilege abuse; task-scoped access	OWASP Agentic Top 10 [49]	MeshGroup actions per member; sponsor rule; group bindings	Preview in the shell; admission built in thoxwork-core
Least privilege per session; no transitive access	NIST SP 800-207 [53]	Group-scoped planning; cross-site and remote only if all three policy levels allow	Preview

Table 13. Mechanisms against requirements. "Built" names where the mechanism runs today; integration of the Digital Employee stack's mechanisms into the ThoxWork shell is in development.

9 Reading the results

Five cautions are part of the results, and the paper's claims are scoped to survive them.

The E1 to E4 results, in both tiers, are invariant checks, not error rates. A deterministic rule engine either enforces a rule against an enumerated adversary or it does not. Zero leaks over 12,000 attempts, simulated or live, means the rules contain no path around the policy for the attack modes tested; it does not estimate how a production deployment would behave under unenumerated attacks, unrelated implementation bugs, or operational misuse. The zero variance across replicates in the oracle says the same thing: there is no noise in the system to average over. The live replay strengthens the claim in

one specific way: the same bars held against independently written production code, and the harness's one disagreement with the implementation (the credential-scoping defect) was a real bug, found and fixed, which is what a discriminating check is for.

Scaling efficiency above 1.0 is a model artifact. The reported E5 efficiencies (1.50 at team size 4, 1.74 at 8, 1.32 at 16, against the 2-member baseline) are superlinear, and superlinear speedup is not a claim this paper makes. Three construction choices produce it: arrivals are a single batch at tick 0, so small teams spend their early ticks queue-bound in a way that flatters larger teams' per-member figures; the only coordination cost in the model grows with group count, not with team size; and the 2-member baseline dedicates one of its two members to review rather than execution. What survives these artifacts is exactly what the pre-registered bar tested: no cell collapses, every cell completes, and per-member throughput at 16 members stays far above half the baseline. The honest reading of E5 is non-collapse under stated assumptions, plus one genuinely informative signal, the emergence of the reviewer bottleneck at team size 16, which matches the consent-fatigue and escalation-capacity concerns in the governance literature [12] and argues for reviewer capacity to be planned as a first-class resource in any real deployment.

Zero-tick denial is a definition, not a speed. E3's deny latency of 0 ticks records that admission is a synchronous decision in the model, taken in the tick of evaluation. Ticks are abstract; the wall-clock cost of admission in the live system is a different quantity, bounded by the decision costs of Section 7.10 for the planner and kit, and not separately benchmarked for the core at this milestone.

The E3 cliff is the rule, drawn. The step between 1.00x and 1.01x in Figure 3, and its live counterpart between 120 and 121 RU in E11, is the binary placeability comparison in Algorithm 1, not a discovered phase transition. Its value is confirmatory: the implemented rule is the specified rule, with no softening at the boundary, no partial credit, and no quiet overflow. The WAIT path of Algorithm 1 is exercised in E1 and E5 and, live, in E11's parallel test; by the drained-group design it is not exercised in E3, and the paper does not present E3 as evidence about queueing behavior. E8 Phase 2 is the edition's queueing evidence.

Agreement is not correctness, and a port is not the rule. E6 and E7 show that independent implementations of the same documented rules agree; a rule wrong in the same way in both would also agree, which is why the rules themselves are printed in full in Section 4 for any reader to falsify. E8 makes the complementary point about measurement hygiene: a divergence figure is a property of the two implementations compared and of the demand model used, and quoting the port's figures as measurements of Algorithm 1 would misstate both. Every divergence number in this paper names its arms.

10 Product implications

PRODUCT IMPLICATIONS

For a company adopting DigitalHumans, the buying question is not which model, but which rules. The model in this paper reduces to five questions a buyer can ask any platform, including this one: Can a task touch only what its team is approved to touch, checked when it runs and not only when it is created? Is oversized work refused out loud, with a reason, instead of quietly rerouted? Does every change of hands leave a written, checkable note? Can anything reach Done without an eligible human's approval on the record? And does the evidence for all four answers live in the customer's own workspace? ThoxWork is designed so that each answer is a mechanism, not a promise, and the lab and the live replay in this paper exist so the mechanisms can be exercised before they are trusted.

Three implications follow for THOX's own roadmap, stated as positions rather than commitments. First, **reviewer capacity is a resource**: E5's bottleneck signal and the governance literature agree that human review is the scarcest element of the system, so MeshGroups should plan reviewer membership the way they plan any approved resource (Section 13, D6). Second, **department scale is the design point**: the orchestration evidence [30] and the positioning of Table 1 both point to bounded teams of tens, federated, over fleets of hundreds, discovered globally. Third, **the interface is the product**: Rolodex's three-verb loop (assign, check in, review) is deliberately small, because the MIT CISR and Work Trend evidence says value comes from redesigned work and team standards [10, 9], and a small, exact loop is what a team can actually standardize on. Delivery follows the positioning in THOX records: ThoxWork at the office and at home, on ThoxDevices and compatible hosts or a desktop application, with the enterprise platform itself released when THOX says it is ready and not before.

11 Security considerations

This section states considerations at the level of public standards and the model's public rules; the full threat model is Section 3 and the security argument is Section 4.8. THOX's internal security configurations are out of scope.

The OWASP Top 10 for Agentic Applications (2026) enumerates the risk classes any agent platform must answer, from goal hijacking through rogue agents [49]. The ThoxWork model's public rules answer several of them structurally: per-group resource approval and double-evaluated admission bound what an agent can be made to touch (excessive agency, tool misuse); the review gate bounds what an agent can commit (improper output handling, human-review failure modes); the handoff chain and append-only event record provide the accountability substrate (repudiation). Other classes, including memory poisoning and inter-agent communication security, are not answered by anything in this paper and are named here as open obligations for any production edition. NIST's agent identity work identifies five focus areas (identification, authorization, delegation, logging and transparency, data flows) and warns that human escalation degrades under consent fatigue [12]; the model's risk-tiered review and its treatment of review as a scarce, planned resource are the design responses

offered here. Persistent, named DigitalHuman personas carry a specific risk identified in the personality-pairing literature: profile information can shape user trust and behavior, so Rolodex persona presentation must remain disclosed and factual, a constraint THOX's colleague standard also imposes [40, 75].

12 Limitations

1. **Simulation, not deployment, for the oracle tier.** The lab proves the rules as modelled enforce the invariants. It does not prove that a production implementation enforces them; that is what E11 tests, at pilot scope. Every oracle number is SIMULATED, and the paper's product claims are scoped by Table 6, including the platform's Coming Soon status in THOX's own copy.
2. **One live implementation, one host.** E11 ran against a single pilot deployment of thoxwork-core on one host with a single database. No fleet behaviour, failover, multi-node execution or customer load was measured, and no E11 figure is a production benchmark.
3. **Preview is not enforcement.** The shell's MeshGroup planner decides but does not authorize or execute. Its agreement with the lab (E6) shows the rules are implemented as documented, not that a deployment enforces them; enforcement claims rest on thoxwork-core and E11 only.
4. **Agreement is not correctness.** The lab planner was written from the same documented rules the product implements; a rule wrong in both would agree. The reference-kit comparison (E7) is against a second, independent implementation, which is stronger. E6 and E7 report the companion review's runs: their delivered result files were read and cross-checked for this edition (headline counts, outcome mixes and commit pins agree with the files), but the runs were not re-executed, because the product planner and kit sources are THOX-internal and were not part of this edition's artifact set. E8, E9 and the port-study reproduction were executed and independently re-verified for this edition.
5. **Enumerated adversaries only.** E1 tests three attack modes, E4 six bypass classes, and E9 the attack classes of its matrix. An unenumerated bypass is not excluded by these results. Propositions 1 and 2 bound what any in-band check can do; they do not cover a compromised verifier or a stolen session key.
6. **Checker independence is definitional, not organizational.** The oracle's checker is independent code that reads only the log and final records, but it shares this paper's definitions (Definitions 3 and 4) with the engine; an error in a definition would pass both. The mitigations are the ones applied here: the definitions are printed in full in Section 4 so any reader can falsify them, the E4 control arm demonstrates the checker distinguishes gated from ungated regimes, and E11's three judges are separate code paths, one of them a separate binary on a read-only database role.
7. **E5 simplifications.** Batch arrivals at tick 0, cooperative reviewers, no failures, no offline members, and no members shared across groups in the grid. Real enterprise load differs on all five. The coordination cost model grows with group count only. Section 9's discounting of the efficiency figures is part of the result.

8. **Abstract units and illustrative demand.** Capacities and demands are abstract RU; no mapping to real compute, storage, latency or cost is claimed, and the lab's resource types are synthetic. E8's shares, and the port study's counts, depend on their stated demand models; the subset relations and deviation decompositions do not.
9. **Harness simplifications in E1 to E4.** In the oracle, tasks are processed sequentially (E1, E4) and concurrency ceilings are raised where concurrency is not the variable under test (E2, E3), so that capacity and concurrency never confound the policy, handoff, admission and review variables. In the live replay, three adaptations apply (Section 6.3): admission is exercised at assignment, the WAIT path is evidenced by the parallel test rather than the sequential drivers, and E2 tasks are submitted to free the pool.
10. **M1 scope limits in the live implementation.** Live dispatch is an unconfigured stub; autonomy tiers are carried and displayed but tier-based action-class gating is not enforced; per-principal HMAC secrets with short-lived tokens stand in for asymmetric host keypairs; the write path is serialized on a single connection; and a public TLS step still needs a domain decision. Each is a named milestone in Section 13, not a footnote.
11. **Partial test coverage.** Swift, Docker-based and PostgreSQL-backed suites did not run for the census (Table 7), the Python runtime's suite was parsed from a review run rather than rerun, and RolodexGo's 292 tests are TypeScript and domain suites, not shipped-client coverage. The hosted guest-demo adapter was not re-run.
12. **Literature transfer and abstract-based positioning.** The cited studies concern other systems and populations; they are used as design warrants, and their effect sizes are not claimed for ThoxWork or its users. Table 1 judges other systems from their abstracts; full texts could show more than the abstracts state. No peer-reviewed head-to-head comparison of local-first versus central-cloud agent-team platforms exists in the surveyed window; the local-first position is argued as a principled, data-control choice with lab support for its workflow rules, not as a proven performance winner.
13. **No customer evidence.** There are no customer deployments, testimonials or outcome metrics in this paper, because none exist in the record to cite.
14. **Interested party.** The authors are THOX principals and the code reviewed is THOX's own. The review process reported defects, including the three census failures and the E11 credential-scoping defect; defects not yet fixed are tracked internally and are not detailed here (Appendix D).

13 Decisions and roadmap

The companion review closed with six founder decisions. This edition dispositions them on the evidence now available; a decision recorded here is the paper's ruling, with its decider and reversal path in the pack's handoff record.

Id	Decision	Disposition
D1	Which MeshGroup admission definition governs (Table 5)	**Decided: pooled admission (Algorithm 1) governs.** Decided by implementation: thoxwork-core implements it and E11 exercised it live, including the WAIT path. The kit remains a reference implementation and comparator. The planner's preview and the core's admission must never be presented as the same decision. Section 4.5 states the full disposition.
D2	Upstream credit for the ThoxWork shell	**Adopted as standing posture:** upstream license notices are kept beside THOX's and OpenWork and OpenCode are credited in every distribution; counsel reviews third-party license terms before any release.
D3	Evidence chain	**Adopted:** the keyed, anchored chain of Equation 2 is the platform's evidence design, written by the workspace, anchored with a second party. thoxwork-core implements the hash-chained, insert-only structure at M1; key management and external anchoring land in the M4 hardening milestone.
D4	Assignment vocabulary	**Adopted:** Table 4 is published as the mapping; the gateway aligns its rejection mapping to blocked, as the contract states. Engineering task in M2.
D5	Customer-facing roster	**Open:** pick one roster and one name for the workspace assistant across products and the website. Founders' call; no engineering is blocked on it at M1 scope.
D6	Reviewer capacity	**Adopted:** reviewers per MeshGroup are planned as a resource, as E5's bottleneck signal and the governance literature both direct.

Table 14. Founder decisions from the comprehensive review, with this edition's dispositions.

Roadmap. M2: land the RolodexGo Teams and Work changeset in the shipped client, wire the native clients to thoxwork-core, and align the gateway's rejection mapping (D4). M3: live dispatch from admission to MeshStack and ThoxRoute execution, honouring pooled admission under the rule of Section 4.5: a dispatch shortfall is a dispatch outcome, reported as such, never a retroactive admission change; carry the sponsor rule into the Digital Employee gateway. M4: an E5-shaped load test against the live system, SAML and SCIM provisioning, an installer, a backup and restore drill, and the D3 key-management and anchoring hardening; and, with people in the loop, measure approval-gate latency and approval-record completeness, which no verified study has done for multi-agent enterprise work. One standing rule binds every milestone: the SIMULATED E5 numbers are never quoted as product targets, capacity plans or marketing figures; live targets are set from live measurements only. ThoxWork remains Coming Soon in THOX's product copy, with no price or date stated here.

14 Provenance, consent and document control

Evidence sources, by tier. Simulated: `exp/results/summary.json` (schema `thoxwork-summary/1.0`), produced by `exp/thoxwork_experiments.py` v1.0.0, master seed 20261104, SHA-256 `f7c72f64b9b8c6f2e1663fbb014374858b2251f2aeaa7d989393a6cdb071dd79`; independently re-run and re-read by a verifier who did not produce it, byte-identical, with all five protocol bars re-checked from the numbers, and re-verified byte-identical again for this edition. Measured, code and lab: the comprehensive companion review's delivered artifact set (planner agreement, kit agreement and timing, divergence study with its scenario file, product test census), audited claim by claim before any number was carried into this edition; the divergence study was re-executed from the delivered

artifacts for this edition and reproduced exactly (Appendix A). This edition's own runs: `exp/results/summary-v110.json`, carrying E8 and E9 in full, produced under the seed discipline of Appendix A and independently re-verified before their numbers entered this paper (verifier report in the artifact bundle, `wp/qa/`). Measured, live: the thoxwork-core acceptance record (run tag `f182809`), retained in the ThoxWork implementation archive with its per-experiment results and drill records. Product status: a repository and records audit covering 44 GitHub repositories and 9 local product records, conducted read-only for the first edition, extended by the companion review's code audit of 10 code bases at the commits in Table 7 and by the M1 milestone record. Literature: a verified survey of recent six and twelve month windows plus labelled foundational works, from which the references below are drawn; every correction in Appendix C names the source it was checked against. Figures 3 to 5 are generated from `summary.json` by `wp/figures.py`; Figures 1 and 2 are diagrams of the stated model.

Trade-secret boundary. This paper discloses the model's public rules and no more. It contains no internal hostnames, network details, credentials, private source code, unfiled intellectual-property particulars beyond the standing patent-pending posture, pricing, customer identities, or DigitalHuman system instructions. MeshStack internals are cited from the published record only. Appendix D states the full disclosure boundary.

Authorship and consent. Authors: Craig Ross, Chief Executive Officer, and Tommy Xaypanya, CTO and Co-Founder (corresponding author), THOX.ai LLC. Publication proceeds under the THOX Research Paper Process v1.0 [76] (Zenodo first, then the one-pass association sync), with each named author's consent to the exact bytes of this edition recorded in the pack's HANDOFF record. No venue submission is made by this edition.

Document control. Version v1.1.0, 2026-11-04. Changes from v1.0.1: comprehensive expansion. New: the literature and positioning of Section 2 in full; the threat model (Section 3); the gateway, approval, lifecycle, planner and evidence material of Sections 4.2 to 4.8, including the five-state-machine reconciliation and the disposition of the admission question D1; the component census and lineage of Section 5; the three-instrument method of Section 6; measurement families E6 to E11 (Sections 7.6 to 7.12), of which E8 and E9 are new runs under this paper's semantics and E11 is the first live replay; the governance mapping (Section 8); the decisions and roadmap (Section 13); and Appendices A to D. Corrected: the errata of Appendix C, applied throughout. Unchanged: the model of Sections 4.1 and 4.5 (Definitions 1 to 5, Algorithm 1, invariants I1 to I5), the pre-registered bars, and every simulated result of Sections 7.1 to 7.5. DOI: 10.5281/zenodo.23965699 (v1.1.0). Concept DOI: 10.5281/zenodo.23963935 (all versions). Prior version: v1.0.1, DOI 10.5281/zenodo.23963936. Companion artifacts: interactive demo (the ThoxWork Virtual Lab, canonical-semantics edition), artifact bundle with experiment code, results, figures, QA report and this paper's shipped files.

A Appendix A. Reproducibility

Oracle (E1 to E5). `python3 exp/thoxwork_experiments.py` writes `exp/results/summary.json`; the suite's self-test re-runs the experiments and asserts byte-identical serialized results. Master seed 20261104. The file's SHA-256 is in Section 14.

Companion runs (E6, E7, E10, census). Host: 2 vCPU Intel Xeon at 2.10 GHz, 8 GB RAM, Linux, Node v22.23.4, Python 3.13.18. Timed configurations had one warm-up run and 30 measured repetitions; figures report medians. The THOX reference admission kit and the ThoxWork planner are THOX source and are not redistributed; the experiments that compare against them record the THOX outcomes in their result files so the comparison can be re-checked without the source.

Port-study reproduction (Section 7.8). `node exp/admission_divergence.js` (seed 20261109), with the lab admission module extracted byte-faithfully from the delivered Evidence Lab single-file HTML (its MeshGroup admission script block, a self-contained UMD module; the delivered artifact set contains no standalone module file) and the delivered scenario file `exp/results/meshgroup_cases.json` (200 scenarios, 50 nodes, 100 requests). The re-execution's counts are identical to the delivered `admission_divergence.json` in every field, including the 56.3 percent share and the 1,655 / 2,678 split of the 4,333.

E8. `python3 exp/e8_divergence.py` writes `exp/results/e8_divergence.json`. Seeds follow the oracle family convention: seed(E, rep) = 20261104 + 10000·E + rep, with E = 8 (Phase 2 offset +5000). World: 12 devices, capacities in {60, 120, 240} RU, each of 5 approved models hosted with probability 0.55 and always by at least two devices; snapshot use per device uniform on [0, 0.95]. Demand model: 50 percent U[5, 60], 30 percent U[61, 240], 12 percent U[241, 480], 8 percent U[481, 1200] RU; 12 percent of requests ask an unapproved resource. Phase 1: 25 replicates × 800 probes. Phase 2: 25 replicates × 400 arrivals plus 400 drain ticks; admitted durations uniform on [5, 40] ticks; the WAIT queue is re-evaluated each tick in FIFO order with a non-blocking scan.

E9. `python3 exp/e9_detection_matrix.py` writes its result file; a supplement covers the suffix-rebuild attack. Equation 2 is implemented from the equation alone: canon is JSON with sorted keys, separators (",", ":"), UTF-8, over the fields (i, t, κ, a, π, previous hash); the genesis link is SHA-256 of a fixed label; the HMAC key is a fixed test key stated in the script, not a production key. Chains of 1,000 records; 500 trials per attack; seeds 20261104 + 10000·9 + trial. Known-answer tests: SHA-256 per the FIPS 180-4 examples (4 vectors) and HMAC-SHA-256 per RFC 4231 cases 1, 2 and 6: 7 of 7 pass. Timings are host measurements and are not part of the reproducibility claim.

Merge discipline. `python3 exp/merge_summary_v110.py` folds the E8 and E9 outputs into `exp/results/summary-v110.json` and asserts, before and after the merge, that the oracle's `summary.json` still hashes to the value in Section 14. The original file is never written by any v1.1.0 tooling.

E11. The acceptance harness drives the deployed thoxwork-core API with the oracle's E1 to E4 generators ported one-to-one (seed formula of Section 6.3, R = 20), plus the 50-way parallel admission test; scoring by the S7 verifier API, the S7 verifier CLI and the harness checker. Run tag

`f182809`. The harness and the service are THOX source and are not redistributed; the per-experiment results are retained in the implementation archive.

B Appendix B. Notation

Symbol	Meaning
RU	Resource unit: the abstract capacity and demand unit of Definitions 1 and 5
capacity(P, r), free(P, r)	Total, resp. currently free, RU over the devices of MeshGroup P that run resource r
g_a, h_r, σ_a	External-effect gate of assignment a, human-approval flag of responsibility r, sensitivity of a
$\alpha_e(a)$, $\alpha_r(a)$	Autonomy level of the employee and of the role on assignment a
h_i, τ_i, k	Hash and HMAC tag of record i, session key
t_i, κ_i, a_i, π_i	Time, kind, actor and payload of record i
(n, h_{n-1})	Anchor: record count and head hash held by a second party

C Appendix C. Errata register

The first edition was checked against live sources and the code for this edition. Every item below is corrected in v1.1.0; none changes its simulated results.

Item in v1.0.0 / v1.0.1	Finding	Source checked	Disposition
Reference [2] title (DynaFlow)	The paper is titled "DynaFlow: Learning Free-Form Multi-Agent Orchestration"	arXiv 2610.34467 [31]	Applied
Reference [3] title (CHAP)	The paper is titled "Collaborative Human-Agent Protocol (CHAP)"	arXiv 2606.09751 [41]	Applied
Local-first reference authors	The fourth author is Mark McGranaghan	ACM DOI 10.1145/3359591.3359737 [71]	Applied
All references	Author lists were surnames only; full lists in order are required	arXiv API, venue pages	Applied (References)
Ju and Aral figures	The current version reports 2,234 participants and 11,024 ads; the first edition quoted an earlier version	arXiv 2503.18238v4 [39]	Applied (Section 2.3)
Orchestration finding	The study says a larger discovery space may contribute; the first edition stated that scale dominates failure	arXiv 2606.20058 [30]	Applied (Section 2.2)
SailPoint 44 percent	The survey reports policies to secure agents, not policies governing them	SailPoint release [8]	Applied (Section 2.1)
Work Trend Index	The two-to-one figure is permutation importance for self-reported impact, associational, not variance explained	Microsoft WorkLab [9]	Applied (Section 2.1)

Item in v1.0.0 / v1.0.1	Finding	Source checked	Disposition
MCP and A2A	Both specifications are now stewarded under the Linux Foundation	Specification sites [17, 18]	Applied (Section 2.1)
MeshGroup implementation claim	The kit cited as the measured reference implementation uses a different admission rule than Algorithm 1 (Table 5), and the first edition's framing did not separate the kit's rule from the paper's rule	THOX reference kit, 8 of 8 tests; thoxwork-core, Section 7.11	Corrected and dispositioned (Sections 4.5, 6.2, 7.7, 7.8; D1)
Lineage	The first edition did not mention that the ThoxWork shell is built on OpenWork and OpenCode, or the runtime on Hermes Agent	Repository comparison [72, 73, 74]	Applied (Sections 5, 5.2)
RolodexGo Teams and Work tabs (corrected in v1.0.1)	The tabs are a built, test-covered changeset of 2026-10-01 that never landed in the shipped client; v1.0.0 presented them as part of the released client	Repository main branch and changeset record	Corrected in v1.0.1 and carried verbatim in substance (Section 5)

Table 15. Errata register for the ThoxWork research paper v1.0.x, with dispositions in this edition.

D Appendix D. Disclosure boundary

The paper states rules and results. It omits internal hostnames, ports, project and database identifiers, credentials, per-role autonomy settings and approver mappings, persona instructions, model-to-base mappings beyond what THOX has published, prices, dates, customer and partner identities, and security defects that are not yet fixed, which were reported to THOX internally. Lineage and license facts are stated because the licenses require attribution.

Conflicts: the authors are THOX principals; this is an interested-party technical account.

References

[1] Craig Ross, Tommy Xaypanya. ThoxWork: One Workplace, One Connected Team. A Policy-Bounded DigitalHuman Work Platform for Enterprise Use. THOX.ai Research technical report v1.0.1, 2026-11-04; Zenodo DOI 10.5281/zenodo.23963936. https://doi.org/10.5281/zenodo.23963936

[2] Craig Ross, Tommy Xaypanya. THOX MeshStack: Capacity-Priced Pipeline Sharding, Grant-Gated KV Transfer and Lease-Fenced Device Roles for a Private Household Mesh. THOX.ai Research technical report v1.0.1, preprint, 2026-11-03; Zenodo DOI 10.5281/zenodo.23919580. https://doi.org/10.5281/zenodo.23919580

[3] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, Graham Neubig. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. NeurIPS 2025 Datasets and Benchmarks Track (poster); arXiv:2412.14161. https://arxiv.org/abs/2412.14161

[4] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. NeurIPS 2024 Datasets and Benchmarks Track (poster); arXiv:2404.07972. https://arxiv.org/abs/2404.07972

[5] Shunyu Yao, Noah Shinn, Pedram Razavi, Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. ICLR 2025 (poster); arXiv:2406.12045. https://arxiv.org/abs/2406.12045

[6] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, Ion Stoica. Why Do Multi-Agent LLM Systems Fail? NeurIPS 2025 Datasets and Benchmarks Track (spotlight); arXiv:2503.13657. https://arxiv.org/abs/2503.13657

[7] Alex Singla, Alexander Sukharevsky, Bryce Hall, Lareina Yee, Michael Chui, Tara Balakrishnan. The state of AI in 2025: Agents, innovation, and transformation. McKinsey & Company (QuantumBlack, AI by McKinsey), November 5, 2025. https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-2025

[8] SailPoint, Dimensional Research. SailPoint research highlights rapid AI agent adoption, driving urgent need for evolved security. SailPoint press release, Austin, TX, May 28, 2025; report 'AI agents: The new attack surface' (survey by Dimensional Research). https://www.sailpoint.com/press-releases/sailpoint-ai-agent-adoption-report

[9] Microsoft WorkLab. Agents, human agency, and the opportunity for every organization (2026 Work Trend Index Annual Report). Microsoft WorkLab, May 5, 2026; survey by Edelman Data x Intelligence. https://www.microsoft.com/en-us/worklab/work-trend-index/agents-human-agency-and-the-opportunity-for-every-organization

[10] Peter Weill, Stephanie L. Woerner. Leveraging Digital Colleagues for Enterprise Value. MIT CISR Research Briefing No. XXVI-4, April 16, 2026. https://cisr.mit.edu/publication/2026_0401_DigitalColleagues_WeillWoerner

[11] National Institute of Standards and Technology, Center for AI Standards and Innovation (CAISI). AI Agent Standards Initiative. NIST initiative page; announced by CAISI on February 17, 2026; page last updated August 14, 2026. https://www.nist.gov/artificial-intelligence/ai-agent-standards-initiative

[12] Harold Booth, William Fisher, Ryan Galluzzo, Joshua Roberts. Accelerating the Adoption of Software and AI Agent Identity and Authorization. NIST National Cybersecurity Center of Excellence, draft concept paper (Initial Public Draft), February 5, 2026; comments closed April 2, 2026. https://csrc.nist.gov/pubs/other/2026/02/05/accelerating-the-adoption-of-software-and-ai-agent/ipd

[13] Josh Woodruff. The Agentic Trust Framework: Zero Trust Governance for AI Agents. Cloud Security Alliance blog, February 2, 2026. https://cloudsecurityalliance.org/blog/2026/02/02/the-agentic-trust-framework-zero-trust-governance-for-ai-agents

[14] Josh Woodruff. Agentic Trust Framework Specification. Version 0.9.1, Public Review Draft, last updated April 2026; GitHub massivescale-ai/agentic-trust-framework; stewardship transferring to CSAI Foundation (CSA press release, April 29, 2026). https://github.com/massivescale-ai/agentic-trust-framework/blob/main/SPECIFICATION.md

[15] Anthropic. Introducing the Model Context Protocol. Anthropic news, November 25, 2024. https://www.anthropic.com/news/model-context-protocol

[16] Rao Surapaneni, Miku Jha, Michael Vakoc, Todd Segal. Announcing the Agent2Agent Protocol (A2A). Google Developers Blog, April 9, 2025. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[17] Model Context Protocol a Series of LF Projects, LLC. Model Context Protocol Specification, revision 2026-07-28. modelcontextprotocol.io; current protocol version 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28

[18] A2A Protocol Project (Linux Foundation). Agent2Agent (A2A) Protocol Specification. a2a-protocol.org, Latest Released Version 1.0.0; project under the Linux Foundation-directed Agentic AI Foundation (blog, August 27, 2026). https://a2a-protocol.org/latest/specification/

[19] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, Bernard Ghanem. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. NeurIPS 2023; arXiv:2303.17760. https://arxiv.org/abs/2303.17760

[20] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024 (oral, per OpenReview); arXiv:2308.00352. https://arxiv.org/abs/2308.00352

[21] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, Maosong Sun. ChatDev: Communicative Agents for Software Development. ACL 2024 (Long Papers), Anthology ID 2024.acl-long.810; arXiv:2307.07924. https://arxiv.org/abs/2307.07924

[22] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. COLM 2024; arXiv:2308.08155. https://arxiv.org/abs/2308.08155

[23] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, Jie Zhou. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. ICLR 2024 (poster); arXiv:2308.10848. https://arxiv.org/abs/2308.10848

[24] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. Proceedings of UIST 2023, ACM; arXiv:2304.03442. https://arxiv.org/abs/2304.03442

[25] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, Ji-Rong Wen. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science 18(6):186345, 2024; DOI 10.1007/s11704-024-40231-1; arXiv:2308.11432. https://arxiv.org/abs/2308.11432

[26] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Tao Gui. The Rise and Potential of Large Language Model Based Agents: A Survey. arXiv preprint arXiv:2309.07864, 2023. https://arxiv.org/abs/2309.07864

[27] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, Xiangliang Zhang. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. IJCAI 2024 Survey Track, pp. 8048-8057, 2024; DOI 10.24963/ijcai.2024/890; arXiv:2402.01680. https://arxiv.org/abs/2402.01680

[28] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, Hoang D. Nguyen. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. arXiv preprint arXiv:2501.06322, 2025. https://arxiv.org/abs/2501.06322

[29] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, Saleema Amershi. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. arXiv preprint arXiv:2411.04468, 2024. https://arxiv.org/abs/2411.04468

[30] Harsh Rao Dhanyamraju, Leonidas Raghav, Aaron Lee. Autonomous Event-Driven Multi-Agent Orchestration for Enterprise AI at Scale. arXiv preprint arXiv:2606.20058, 2026 (v2, 28 Sep 2026). https://arxiv.org/abs/2606.20058

[31] Ruhao Liu, Qi Li, Shizun Wang, Xinchao Wang. DynaFlow: Learning Free-Form Multi-Agent Orchestration. arXiv preprint arXiv:2610.34467, 2026 (v2, 29 Oct 2026). https://arxiv.org/abs/2610.34467

[32] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, Alexandre Lacoste. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? ICML 2024 (poster); arXiv:2403.07718. https://arxiv.org/abs/2403.07718

[33] Kung-Hsiang Huang, Akshara Prabhakar, Sidharth Dhawan, Yixin Mao, Huan Wang, Silvio Savarese, Caiming Xiong, Philippe Laban, Chien-Sheng Wu. CRMArena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments. NAACL 2025; arXiv:2411.02305. https://arxiv.org/abs/2411.02305

[34] Abubakarr Jaye, Nigel Boachie Kumankumah, Chidera Biringa, Anjel Shaileshbhai Patel, Sulaiman Vesal, Dayquan Julienne, Charlotte Siska, Manuel Raúl Meléndez Luján, Anthony Twum-Barimah, Mauricio Velazco, Tianwei Chen. CORPGEN: Simulating Corporate Environments with Autonomous Digital Employees in Multi-Horizon Task Environments. arXiv preprint arXiv:2602.14229, 2026. https://arxiv.org/abs/2602.14229

[35] Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Matheus Kunzler Maldaner, Madeleine Grunde-McLaughlin, Eric Zhu, Griffin Bassman, Jacob Alber, Peter Chang, Ricky

Loynd, Friederike Niedtner, Ece Kamar, Maya Murad, Rafah Hosn, Saleema Amershi. Magentic-UI: Towards Human-in-the-loop Agentic Systems. arXiv preprint arXiv:2507.22358, 2025. https://arxiv.org/abs/2507.22358

[36] K. J. Kevin Feng, David W. McDonald, Amy X. Zhang. Levels of Autonomy for AI Agents. Knight First Amendment Institute 'AI and Democratic Freedoms' essay series, 2025; arXiv:2506.12469. https://arxiv.org/abs/2506.12469

[37] K. J. Kevin Feng, Kevin Pu, Matt Latzke, Tal August, Pao Siangliulue, Jonathan Bragg, Daniel S. Weld, Amy X. Zhang, Joseph Chee Chang. Cocoa: Co-Planning and Co-Execution with AI Agents. CHI 2026; arXiv:2412.10999. https://arxiv.org/abs/2412.10999

[38] Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, Diyi Yang. Collaborative Gym: A Framework for Enabling and Evaluating Human-Agent Collaboration. ICLR 2026 (poster); arXiv:2412.15701. https://arxiv.org/abs/2412.15701

[39] Harang Ju, Sinan Aral. Collaborating with AI Agents: Field Experiments on Teamwork, Productivity, and Performance. arXiv preprint arXiv:2503.18238, 2025 (v4, 15 Oct 2026). https://arxiv.org/abs/2503.18238

[40] Harang Ju, Sinan Aral. Personality pairing improves human-AI collaboration. arXiv preprint arXiv:2511.13979, 2025 (v4, 28 Aug 2026). https://arxiv.org/abs/2511.13979

[41] Arsalan Shahid, Gordon Suttie, Philip Black. Collaborative Human-Agent Protocol (CHAP). arXiv preprint arXiv:2606.09751, 2026 (v2, 20 Jun 2026). https://arxiv.org/abs/2606.09751

[42] Elham Tabassi. Artificial Intelligence Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology, NIST AI 100-1, January 26, 2023; DOI 10.6028/NIST.AI.100-1. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

[43] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, Kamie Roberts. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. National Institute of Standards and Technology, NIST AI 600-1, July 26, 2024; DOI 10.6028/NIST.AI.600-1. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence

[44] ISO/IEC JTC 1/SC 42. ISO/IEC 42001:2023 Information technology - Artificial intelligence - Management system. International Standard ISO/IEC 42001:2023, Edition 1.0, 2023-12-18. https://webstore.iec.ch/publication/90574

[45] European Parliament and Council. REGULATION (EU) 2024/1689 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 13 June 2024 laying down harmonised rules on artificial intelligence and amending Regulations (EC) No 300/2008, (EU) No 167/2013, (EU) No 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and Directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (Artificial Intelligence Act). Official Journal of the European Union, L series, 2024/1689, 12.7.2024; Regulation (EU) 2024/1689; ELI http://data.europa.eu/eli/reg/2024/1689/oj. https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng

[46] European Parliament and Council. REGULATION (EU) 2026/1744 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 8 July 2026 amending Regulations (EU) 2024/1689, (EU) 2018/1139 and (EU) 2023/1230 as regards the simplification of the implementation of harmonised rules on artificial intelligence. Official Journal of the European Union, OJ L 2026/1744, 24.7.2026; Regulation (EU) 2026/1744 (Digital Omnibus on AI). https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L_202601744

[47] OWASP Gen AI Security Project. OWASP Top 10 for LLM Applications 2025. OWASP Gen AI Security Project, resource page dated November 17, 2024; entries LLM01:2025 and LLM06:2025. https://genai.owasp.org/llm-top-10/

[48] OWASP Agentic Security Initiative (Gen AI Security Project). Agentic AI - Threats and Mitigations. OWASP Gen AI Security Project, February 17, 2025. https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/

[49] OWASP Gen AI Security Project - Agentic Security Initiative. OWASP Top 10 for Agentic Applications for 2026. OWASP Gen AI Security Project, resource page dated December 9, 2025 (PDF: Version 2026, December 2025); entries ASI01-ASI10. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[50] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), pp. 79-90, 2023; DOI 10.1145/3605764.3623985; arXiv:2302.12173. https://arxiv.org/abs/2302.12173

[51] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024; DOI 10.52202/079017-2636; arXiv: 2406.13352. https://arxiv.org/abs/2406.13352

[52] Qiusi Zhan, Zhixiang Liang, Zifan Ying, Daniel Kang. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of the Association for Computational Linguistics: ACL 2024, pp. 10471-10506, 2024; DOI 10.18653/v1/2024.findings-acl.624; arXiv:2403.02691. https://aclanthology.org/2024.findings-acl.624/

[53] Scott Rose, Oliver Borchert, Stu Mitchell, Sean Connelly. Zero Trust Architecture. NIST Special Publication 800-207, August 2020; DOI 10.6028/NIST.SP.800-207. https://csrc.nist.gov/pubs/sp/800/207/final

[54] Vincent Hu, David Ferraiolo, Richard Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, Karen Scarfone. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, January 2014 (includes updates as of 08-02-2019); DOI 10.6028/NIST.SP.800-162. https://csrc.nist.gov/pubs/sp/800/162/upd2/final

[55] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, Alex Pentland. Authenticated Delegation and Authorized AI Agents. arXiv preprint arXiv:2501.09674, 2025. https://arxiv.org/abs/2501.09674

[56] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, Markus Anderljung. Visibility into AI Agents. ACM FAccT 2024; arXiv: 2401.13138. https://arxiv.org/abs/2401.13138

[57] Alan Chan, Kevin Wei, Sihao Huang, Nitarshan Rajkumar, Elija Perrier, Seth Lazar, Gillian K. Hadfield, Markus Anderljung. Infrastructure for AI Agents. TMLR 2025; arXiv:2501.10114. https://arxiv.org/abs/2501.10114

[58] Bruce Schneier, John Kelsey. Secure audit logs to support computer forensics. ACM Transactions on Information and System Security (TISSEC), 1999, pp. 159-176; DOI 10.1145/317087.317089. https://doi.org/10.1145/317087.317089

[59] Scott A. Crosby, Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. USENIX Security '09 (Technical Sessions), USENIX Association, 2009. https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf

[60] B. Laurie, A. Langley, E. Kasper. Certificate Transparency. IETF RFC 6962 (Experimental), June 2013; DOI 10.17487/RFC6962. https://www.rfc-editor.org/rfc/rfc6962

[61] B. Laurie, E. Messeri, R. Stradling. Certificate Transparency Version 2.0. IETF RFC 9162 (Experimental), December 2021; DOI 10.17487/RFC9162. https://www.rfc-editor.org/rfc/rfc9162

[62] Luc Moreau, Paolo Missier. PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013; REC-prov-dm-20130430. https://www.w3.org/TR/2013/REC-prov-dm-20130430/

[63] Timothy Lebo, Satya Sahoo, Deborah McGuinness. PROV-O: The PROV Ontology. W3C Recommendation, 30 April 2013; REC-prov-o-20130430. https://www.w3.org/TR/2013/REC-prov-o-20130430/

[64] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. 28th USENIX Security Symposium (USENIX Security 19), Santa Clara, CA, USENIX Association, pp. 1393-1410, August 2019; ISBN 978-1-939133-06-9. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[65] SLSA, The Linux Foundation. Build: Provenance. SLSA specification v1.2, slsa.dev (Linux Foundation); in-toto predicateType https://slsa.dev/provenance/v1. https://slsa.dev/spec/v1.2/build-provenance

[66] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, Rafael Ferreira da Silva. PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows. IEEE e-Science 2025; arXiv:2508.02866. https://arxiv.org/abs/2508.02866

[67] Yi Nian, Aojie Yuan, Haiyue Zhang, Jiate Li, Li Li, Xiyang Hu, Hua Wei, Xiongye Xiao, Chaowei Xiao, Yue Zhao. Auditable Agents. arXiv preprint arXiv:2604.05485 (v4, 27 Sep 2026), 2026; authors state a condensed 5-page version was accepted to ACM AI Summit 2026 (Visionary Papers track). https://arxiv.org/abs/2604.05485

[68] Jeremy Qin, David Schmotz, Derck Prinzhorn, Luca Beurer-Kellner, Ameya Prabhu, Maksym Andriushchenko. LLM Agents Can Easily Tamper With Their Own Traces. arXiv preprint arXiv:2609.30266 (v1, 24 Sep 2026), 2026. https://arxiv.org/abs/2609.30266

[69] National Institute of Standards and Technology. Secure Hash Standard (SHS). NIST FIPS 180-4, August 2015; DOI 10.6028/NIST.FIPS.180-4. https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[70] H. Krawczyk, M. Bellare, R. Canetti. HMAC: Keyed-Hashing for Message Authentication. IETF RFC 2104 (Informational), February 1997; DOI 10.17487/RFC2104. https://www.rfc-editor.org/rfc/rfc2104

[71] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, Mark McGranaghan. Local-First Software: You Own Your Data, in spite of the Cloud. Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '19), Athens, Greece, ACM, pp. 154-178, 2019; DOI 10.1145/3359591.3359737. https://doi.org/10.1145/3359591.3359737

[72] Different AI, Inc. openwork: The open-source alternative to Claude Cowork (powered by opencode). GitHub repository different-ai/openwork; accessed 2026-11-04. https://github.com/different-ai/openwork

[73] anomalyco. opencode: The open source coding agent. GitHub repository anomalyco/opencode, MIT license; accessed 2026-11-04. https://github.com/anomalyco/opencode

[74] Nous Research. hermes-agent: The agent that grows with you. GitHub repository NousResearch/hermes-agent, MIT license; accessed 2026-11-04. https://github.com/NousResearch/hermes-agent

[75] THOX.ai LLC. DigitalHuman Colleague and Disclosure Standard v1.0. THOX.ai internal standard, 2026-11-01.

[76] THOX.ai LLC. THOX Research Paper Process v1.0. THOX.ai internal standard, 2026-11-03.

[77] Craig Ross, Tommy Xaypanya. ThoxOS. THOX.ai Research technical report, 0.2.0 review edition, 2026; Zenodo DOI 10.5281/zenodo.23876230. https://doi.org/10.5281/zenodo.23876230